System Verification and Validation Plan for TeLaC

Team 24, Maca Boca
Javayria Mudassar
Yilin Jiang
Nour Fawaz
Catherine Yang

November 3, 2026

Revision History

Date	Version	Notes
31-10-2026	1.0	Nour: Added sections 2.1, 3.6, parts of 4, 7, 8
02-11-2026	1.1	Everyone: Added the remaining sections and final fixes

Contents

1 Symbols, Abbreviations, and Acronyms **v**

2 General Information **1**
- 2.1 Summary . . . 1
  - 2.1.1 User Interface . . . 1
  - 2.1.2 Audio Analysis Tool (SAFA) . . . 2
  - 2.1.3 Report Generation . . . 2
- 2.2 Objectives . . . 3
  - 2.2.1 Ensure Accuracy of Transcripts and Report Generation 3
  - 2.2.2 Ensure Security and Privacy of Patient Data . . . 3
  - 2.2.3 Demonstrate Usability of the Integrated Workflow . . . 4
  - 2.2.4 Out of Scope . . . 4
- 2.3 Custom Documents . . . 5
  - 2.3.1 Wireframe (Fall) . . . 5
  - 2.3.2 Usability Report (Winter) . . . 6
- 2.4 Relevant Documentation . . . 7

3 Plan **8**
- 3.1 Verification and Validation Team . . . 8
- 3.2 SRS Verification . . . 10
- 3.3 Design Verification . . . 13
- 3.4 Verification and Validation Plan Verification . . . 14
- 3.5 Implementation Verification . . . 16
- 3.6 Automated Testing and Verification Tools . . . 16
- 3.7 Software Validation . . . 17

4 System Tests **18**
- 4.1 Tests for Functional Requirements . . . 18
  - 4.1.1 Data Collection and Storage . . . 19
  - 4.1.2 Video and Audio Analysis . . . 22
  - 4.1.3 Authentication and Accounts . . . 26
  - 4.1.4 Patient Management . . . 30
  - 4.1.5 Transcript Review and Editing . . . 33
  - 4.1.6 Playback and Navigation . . . 37
  - 4.1.7 Assessment Evaluation and Notes . . . 41
  - 4.1.8 Results Display and Reporting . . . 45

4.1.9 Assessment Interface . . . 49
4.2 Tests for Nonfunctional Requirements . . . 50
4.2.1 Usability . . . 51
4.2.2 Understandability . . . 54
4.2.3 Performance . . . 55
4.2.4 Convenience . . . 57
4.2.5 Accuracy . . . 60
4.2.6 Reliability . . . 61
4.2.7 Recording Quality . . . 63
4.2.8 Maintainability . . . 64
4.2.9 Security . . . 65
4.2.10 Compliance . . . 69
4.2.11 Capacity . . . 71
4.3 Traceability Between Test Cases and Requirements . . . 72

5 Unit Test Description **75**
5.1 Unit Testing Scope . . . 76
5.2 Tests for Functional Requirements . . . 76
5.2.1 Module 1 . . . 76
5.2.2 Module 2 . . . 77
5.3 Tests for Nonfunctional Requirements . . . 77
5.3.1 Module ? . . . 78
5.3.2 Module ? . . . 78
5.4 Traceability Between Test Cases and Modules . . . 78

6 Appendix **79**
6.1 Symbolic Parameters . . . 79
6.2 Requirements Feedback Form . . . 79
6.3 Usability Survey Questions . . . 81

7 Appendix — Generative AI Use Disclosure **84**
7.1 AI Tools Used . . . 84
7.2 How and Where Used . . . 84
7.3 Verification of AI-Generated Content . . . 85

8 Appendix — Reflection **86**

List of Tables

1 Symbols, Abbreviations, and Acronyms v
2 Verification and Validation Team Roles and Responsibilities . 8
3 Traceability Between Functional System Tests and Functional Requirements 73
4 Traceability Between Functional System Tests and Use Cases . 74
5 Traceability Between Nonfunctional System Tests and Nonfunctional Requirements 75
6 Variable Names and Values 80

1 Symbols, Abbreviations, and Acronyms

Table 1: Symbols, Abbreviations, and Acronyms

symbol	**description**
AES	Advanced Encryption Standard
API	Application Programming Interface
BAA	Business Associate Agreement
HIPAA	Health Insurance Portability and Accountability Act
SAFA	Stands for Speech Annotation For Application, Access, Accuracy
SLP	Speech Language Pathologist
SRS	Software Requirements Specification
TeLaC	Telehealth Language Assessment for Children
TLS	Transport Layer Security
VnV	Verification and Validation
WCAG	Web Content Accessibility Guidelines

This document outlines how the team will confirm that the TeLaC application satisfies the requirements in the SRS and supports the needs of speech-language pathologists. Section 2 gives an overview of the system and the goals of this plan. Section 3 explains how verification and validation will be carried out. Section 4 defines the system tests for the functional and nonfunctional requirements, and Section 5 describes the unit tests to be run once the detailed design is complete.

2 General Information

This section provides a brief overview of the software being produced and its objectives, how it will be tested and its relevant documentation.

2.1 Summary

The software being tested is TeLaC, a platform for remote speech and language assessment of Mandarin-English bilingual children. It addresses the usability and workflow challenges speech-language pathologists (SLPs) face when conducting remote assessments of bilingual pediatric patients. It does this by integrating the SAFA annotation tool into one platform, auto-generating transcripts, providing a hub for patient records and assessment results, and generating assessment reports. The goal is to help clinicians conduct assessments more efficiently and consistently while supporting ongoing monitoring of a child's speech development, all in one centralized system.

The objective of this VnV plan is to build confidence that TeLaC is usable, accurate, and trustworthy enough to support clinical decisions. TeLaC assists clinicians and does not replace their judgment (SR3). The requirements being verified are defined in the SRS, and the risks that determine test priority are identified in the Hazard Analysis. The system has three main components described below.

2.1.1 User Interface

The user interface must be intuitive and easy to navigate for clinicians with limited technical fluency (SR8, NFR1, NFR2). SLPs use it for the entire workflow, from selecting a patient to downloading a report (FR5, FR9). A

confusing interface slows them down and risks mistakes, so the plan verifies the following:

- Clinicians can complete core tasks, such as assigning a speaker role, viewing results, and adding a patient, without training (NFR1)
- Clinicians can move between workflow stages without losing saved work (FR5, NFR3)
- Clinicians can always tell which content is automatically generated and which they have verified (FR14).

2.1.2 Audio Analysis Tool (SAFA)

SAFA auto-generates transcripts of the assessment recordings and identifies speakers (FR1, FR4). Clinicians then verify and correct these outputs (FR2, FR6). Transcript accuracy is critical considering errors in code-mixed Mandarin-English speech, can lead to wrong conclusions about a child's abilities (SRS Section 9.4). The system is therefore tested to ensure that:

- Transcripts are generated reliably for supported videos (FR1, NFR17)
- The transcript stays synchronized with the recording during playback (FR20, FR21, NFR13)
- Clinician edits and annotations are saved and persist across sessions (FR2, FR19, NFR18)

2.1.3 Report Generation

Report generation compiles a patient's results, the clinician's edits, and notes into a report that the SLP can review and edit before downloading (FR7, SR6). Because the report becomes part of the clinical record and may be relied on by families, schools, and other professionals, its correctness is crucial. The system is therefore tested to ensure that:

- Every change a clinician makes must appear in the report, otherwise the automatically generated content remains as is (FR7)
- Edits to results are reflected correctly and instantly (FR8, NFR4)
- Exports and reports are only accessible to the assigned clinician (NFR5, SRS Section 7.5)

2.2 Objectives

This VnV plan aims to evaluate whether TeLaC meets its key goals for transcript and report correctness, patient data security, and usability of the integrated workflow. Priorities come from the risks identified in the Hazard Analysis. The following subsections define each objective in the context of this project.

2.2.1 Ensure Accuracy of Transcripts and Report Generation

It is important to make sure clinicians can trust the information TeLaC provides to them. Since the report relies on the generated transcripts, the correctness of both aspects is an objective of the project. The system should:

- **Generate transcripts reliably:** The supported assessment videos should produce a transcript at a rate of at least GENERATE_SUCCESS (FR1, NFR17).
- **Distinguish speakers:** Each utterance in a transcript should be labelled with the correct speaker (FR4).
- **Reflect clinician changes in reports:** Every edit and annotation a clinician saves should also appear in the patient record and the generated report (FR7, FR19).
- **Preserve clinician work:** Saved edits should persist across sessions, and data should be backed up regularly and restorable without loss (NFR9, NFR18).

2.2.2 Ensure Security and Privacy of Patient Data

TeLaC stores recordings, transcripts, and health information about children. A breach would harm patients and violate HIPAA, so protecting this data is a key objective of the project. The system should:

- **Restrict access:** Clinicians should only be able to access the patients assigned to them, including through URLs and the API (NFR5).
- **Protect idle sessions:** Sessions should end after IDLE_TIMEOUT minutes of inactivity, and credentials should be required before any patient data is shown again (NFR6).

- **Encrypt patient data:** Recordings and transcripts should be encrypted in transit and in storage, including when sent to third-party transcription services (NFR7).
- **Log access:** Every view, change, download, and export of patient data should be recorded in an audit log that clinicians cannot edit (NFR8).
- **Meet compliance requirements:** The HIPAA technical safeguards should be satisfied, and third-party services handling patient data should be covered by a Business Associate Agreement (NFR15).

2.2.3 Demonstrate Usability of the Integrated Workflow

TeLaC's main value is combining SAFA and the clinician dashboard into one workflow. If the workflow is confusing, clinicians lose time and risk making mistakes, so usability is the final objective of the project. The system should:

- **Provide a user-friendly interface:** The interface should be clear and consistent, with readable text, labelled controls, and a layout that clinicians with limited technical fluency can navigate easily (SR8).
- **Be learnable:** Clinicians with limited technical experience should be able to complete core tasks without training, such as adding a patient, assigning speaker roles, and viewing results (NFR1, NFR2).
- **Support a seamless workflow:** Clinicians should be able to move from selecting a patient to downloading a report without losing work between stages (FR5, NFR3).
- **Make generated content clear:** Clinicians should always be able to tell which content was automatically generated and which they have verified (FR14, NFR14).
- **Be measured through user testing:** Success is measured through usability testing with clinicians, using task completion rate, time on task, and a post-task survey (Appendix).

2.2.4 Out of Scope

Some objectives are out of scope for this VnV plan due to time and resource limitations:

- **Advanced detection of clinical cues:** Detecting subtle behavioural or linguistic signals beyond interference flags and grammar and phonology suggestions is a stretch goal. It will only be verified if implemented, since validating it would require clinical expertise and labelled data the team does not have.
- **Accuracy of third-party speech services:** The speech recognition and keyword detection services (Whisper and Deepgram) are assumed to have been verified by their developers. The plan verifies that TeLaC handles their output correctly and that clinicians can review and correct it, not the services' accuracy on code-mixed speech.
- **Administrative and physical HIPAA safeguards:** Staff training and breach notification procedures are the responsibility of the institutions that use TeLaC, as stated in SRS Section 7.5. Only the technical safeguards are verified.
- **Full UI standards compliance:** The focus is on practical usability for clinicians rather than exhaustive WCAG compliance or visual polish.

2.3 Custom Documents

Two custom documents (CDs) were chosen in the Problem Statement, and both support this VnV plan. Improving SAFA's usability is one of the project's main goals, and many of the requirements in the SRS are about how easily clinicians can use the system (SR8, NFR1, NFR2, NFR14). These are hard to verify with automated tests alone, so the CDs provide the team a structured way to evaluate them with real users. The Wireframe is used first to validate the requirements and the interface design before finalizing the implementation, and the Usability Report is used later to verify that the system actually meets the usability requirements.

2.3.1 Wireframe (Fall)

The Wireframe is a mock-up of SAFA's redesigned interface (although it can also include mockups for the TeLaC application pages), covering the integrated SAFA annotation tool and the main workflow screens (selecting a patient and assessment, choosing a workflow, reviewing and correcting the

transcript, and generating a report). It will be made using a design tool like Figma and presented to the supervisor, the research team, and clinicians. Its role in this plan is validation rather than verification, as it checks that the requirements describe what clinicians actually need before the interface is implemented. During the review, the team will:

- Present each workflow by walking reviewers through the mock-up and ask whether the steps match how they would normally analyze an assessment (SR5)
- Ask reviewers to point out which content is automatically generated and which they have verified, which can be an early check of NFR14
- Get feedback on the overall look and layout of the transcripts, both in the TeLaC view and the SAFA view
- Make note of any labels, icons, or steps that reviewers find unclear (NFR2)
- Record missing features or changes reviewers request

The document will describe the mock-ups and discuss the insights gained due to this review. Any feedback that changes a requirement will be added to the SRS and tracked as its own GitHub issue.

2.3.2 Usability Report (Winter)

The Usability Report verifies the usability requirements against the final system. It will contain the testing plan, the results, and a discussion of the findings and user feedback. This includes any suggested improvements to how the system looks and functions. The testing will be systematic and quantified so that the results can be compared directly against the fit criteria of the usability requirements defined in the SRS. The test plan will include the following (this is subject to change over the course of the project):

- **Participants:** clinicians that will be provided by the research team's contacts who have not been trained on the system
- **Tasks:** core tasks such as adding a patient, assigning a speaker role, correcting a transcript chunk, adding an annotation, and generating

a report. These are the same features tested for correctness in Section 4 (e.g., FR-ST-TR1 and FR-ST-TR2), so the usability sessions only measure how easily clinicians can use them

- **Measures:** task completion rate, time to complete each task, number of times the participant needed help, and whether each function was found on the first attempt
- **Survey:** a post-test questionnaire on ease of use and satisfaction (Appendix 6.3)

The results will also be compared with the standalone SAFA tool, since the Problem Statement measures the usability goal as an improvement over the current SAFA interface. The Usability and Understandability tests in Section 4 will be run as part of these sessions, and the results will be reported in the VnV Report.

2.4 Relevant Documentation

- Problem Statement and Goals: **?**
- Development Plan: **?**
- System Requirements Specification: **?**
- Hazard Analysis: **?**
- Wireframe: **?**
- Usability Report: **?**

The documents listed above are artifacts that will be used to verify and validate the implementation of the TeLaC system. The SRS defines the requirements that the system tests in Section 4 verify, and the Hazard Analysis identifies safety and security requirements covered by those tests.

The Problem Statement and Goals is used to validate that the system meets the needs of speech-language pathologists. The Development Plan details the team roles and tools that support the activities in Section 3.

Finally, the Wireframe will be used to validate the redesigned interface with clinicians, and a final Usability Report will record the results of the team's usability testing in order to verify the usability requirements.

3 Plan

This section outlines how artifacts of the TeLaC project will be verified and validated throughout and after the development process. It introduces the verification and validation team, describes the approaches used to verify the project's documents and implementation, and ends with the validation of the software.

3.1 Verification and Validation Team

The following table displays the roles and responsibilities of each team member.

Table 2: Verification and Validation Team Roles and Responsibilities

Name	**Roles and Responsibilities**
Javayria Mudassar	• Stakeholder Communication Lead • Security Verification • Transcription Verification • SRS Verification
Yilin Jiang	• UI Design Verification • Frontend Verification • Integration Lead • SRS Verification
Nour Fawaz	• Documentation Reviewer • Frontend Verification • Usability Testing Lead • VnV Verification
Catherine Yang	• Code Review Lead • Security Verification • Report Verification • VnV Verification

Name	Roles and Responsibilities
Dr. Irene Yuan	• Providing feedback during project development and connecting the team to speech language therapy researchers
Dr. Yao Du	• Providing feedback on report generation; connecting the team with SLP users for user experience and usability testing
Dr. Yang Yue	• Providing feedback on security during project development
Tanya Djavaherpour	• Milestone and documentation feedback

Role Descriptions:

- **Stakeholder Communication Lead:** Runs validation sessions with supervisors and SLPs, and records their feedback on requirements and system outputs.
- **Code Review Lead:** Reviews pull requests against the requirements and ensures CI checks pass before merging.
- **Documentation Reviewer:** Checks deliverables for consistency with the SRS and VnV Plan, and verifies AI use is disclosed and checked.
- **Security Verification:** Tests access control, session timeouts, encryption, audit logging, backups, and HIPAA compliance.
- **Frontend Verification:** Tests that UI components behave correctly and clinicians can move between workflow stages without losing work.
- **UI Design Verification:** Reviews diagrams and mockups against the requirements before implementation.
- **Transcription Verification:** Tests transcript generation, speaker labelling, and interference and error flags against the team's answer keys.

- **Report Verification:** Tests that reports and exports include all clinician edits and verified results, and match stored data.
- **Usability Testing Lead:** Runs usability tests with clinicians and measures task completion and survey scores against the usability requirements.
- **Integration Lead:** Tests the end-to-end workflow across SAFA and the dashboard to confirm they work as one system.
- **SRS Verification:** Confirms requirements are clear, testable, and traced to at least one test.
- **VnV Verification:** Reviews test cases for completeness, specificity, and traceability.

3.2 SRS Verification

The SRS is verified through four reviews, each suited to catching a different kind of problem. The team first checks its own work for completeness and consistency, the peer review team and TA then check that the document is clear to an outside reader, and finally the supervisor and research team check that it describes what clinicians actually need. Feedback from all four reviews is tracked the same way, which is to create a dedicated GitHub issue for each piece of actionable feedback.

Internal Checklist Review: Before each SRS revision is submitted, every team member reviews the sections written by another member. This way, each section is looked over by someone other than the original author. The reviewer works through the SRS checklist, which covers the template and writing quality, along with the project-specific checklist below. The checklist the team decided on covers problems that are specific to this project. These checks are done in the pull request for each section, which already needs approval from at least one other team member before it can be merged (Development Plan). The reviewer leaves a comment on any item that fails, and the pull request is not approved until it is fixed or the author explains why no change is needed.

- ❑ Every functional requirement traces to at least one stakeholder requirement.

- ❑ Every stakeholder requirement is supported by evidence from the research team's study (SRS Appendix A) or by a supervisor decision.
- ❑ Every functional and nonfunctional requirement has a fit criterion that a tester could check without asking the author what it means.
- ❑ Every threshold in a fit criterion is written as a symbolic constant rather than a fixed number.
- ❑ Every functional and nonfunctional requirement is covered by at least one system test in Section 4.
- ❑ Each requirement states what the system must do, not how it will be built.
- ❑ The use cases, functional requirements, and workflow state diagrams use the same names for the same states and actions.
- ❑ Each of the three clinician workflows (Direct Analysis, Annotation-First, and On-Demand Refinement) can be traced from start to finish through the use cases without a missing step.
- ❑ Automatically generated content (transcripts, scores, and flags) is always treated as a suggestion the clinician can review and correct (SR3).
- ❑ Every requirement that involves patient data states who is allowed to access it (SR7).
- ❑ Requirements refer to the supported assessment languages in general, rather than naming a specific language pair.
- ❑ Domain terms (e.g., code-switching, speaker role, chunk) are defined in the Definitions and Acronyms section before they are used.

Peer and TA Review: The assigned peer review team and the TA read the SRS without any prior knowledge of the application or speech language domain. Since they have no background in speech-language pathology, they are best suited to find sections that are unclear or that assume domain knowledge, which also matters for future developers who will need to maintain the system. Peer reviewers report each problem as a GitHub issue using the team's Peer Review issue template, which records the artifact, the reviewing

team, and a description of the problem. TA feedback from grading documents is turned into issues in the same way.

Supervisor and Research Team Review: The supervisor and research team are the only reviewers who can confirm that the requirements match real clinical practice. Their review is guided by a requirements feedback form, given in full in Appendix 6.2, which links to the functional requirements. The form only covers what the team will build and leaves out the recording features already built by the previous capstone team. It asks whether the requirements reflect how they would ideally review a remote assessment, what is missing, what should be changed or removed, which requirements are essential in a clinical setting, and which 10–15 requirements should be prioritized for the first demo. It also asks them to make decisions the team cannot make on its own: how the generated transcript should be displayed in the assessment interface (SR2), what information a clinical report should be generated from (SR6, FR7), who may view or download recordings (SR7), and which pattern detection features are in scope.

Since the supervisors are busy and more likely to give feedback during scheduled meetings than on their own time, the form is used as the question guide for breakout sessions during the virtual supervisor meetings, organized using the team's Supervisor Meeting issue template. The form is also shared ahead of time, so supervisors can respond in writing if they prefer. During each session, a team member records the answers in the form, so the team keeps a written record of each supervisor's feedback. When an answer is unclear or supervisors disagree, the team walks through the related workflow step by step using its use cases and state diagram, so that they can point out exactly where it differs from their practice. The sessions also cover what the form cannot, such as whether the values proposed for the symbolic constants are realistic (e.g., the task success rate and task time in NFR1 and NFR2). The agreed priority list then decides which requirements are implemented first for the Proof of Concept and Revision 0 demos. Validation with clinicians continues through the Wireframe CD (Section 2.3), where clinicians can provide feedback after seeing visual mockups. This might help them point out any changes that need to be made in a way that is easier than reading written requirements.

Feedback Tracking: Every requested change, whether from the checklist review, peer review, TA, form, or meeting, becomes a GitHub issue assigned to the owner of the affected section. Each issue is closed by a pull request that links to it, or closed with a written reason if the team decides not to make the change. All changes must be recorded in the SRS document's revision history table. An SRS revision is considered verified once all four reviews are complete and every resulting issue has been closed.

3.3 Design Verification

The design of TeLaC will be documented once the design stage begins. Afterwards, design documents will be verified through a combination of internal team reviews, peer and TA reviews, supervisor reviews, and designated design review sessions with the research team and clinicians.

- **Internal team review:** Each design document will be reviewed by at least one team member who did not write it. The reviewer will also check the document against the design checklists provided in the course repository, and any problems found will be logged as GitHub issues. On completion of an interface, a member of the team will also review it to ensure that it follows the Wireframe.

- **Peer and TA review:** The assigned peer review team and the course TA will review the design documents. Their feedback will be logged as GitHub issues, assigned to a team member, and closed with a reference to the commit that resolves it.

- **Supervisor review:** The design will be presented to the supervisor during a scheduled meeting. The team will walk through the module decomposition and the main data flow, from uploading a recording to transcription, SAFA analysis, and report generation, and ask whether the design fits the needs of the research team. If changes are needed, they will be noted and the design documents will be updated before implementation.

- **Stakeholder review:** When presenting the Wireframe of the redesigned interface to our stakeholders (clinicians and researchers), a short questionnaire on layout, navigation, and workflow will be provided for them to fill out. Their responses will be used to revise the

interface before implementation. The questionnaire will be included in the appendix of this document and expanded on in the Usability Report.

During these reviews, the team will also confirm that:

- Every functional requirement in the SRS is assigned to at least one module in the design.
- Each module hides a single design decision that is likely to change, such as the speech recognition model or the database.
- Module interfaces are consistent with each other, and each one has clearly defined inputs, outputs, and exceptions.
- Any mitigation identified in the Hazard Analysis, such as access control and protection of patient data, is handled by a specific module.

3.4 Verification and Validation Plan Verification

The verification and validation plan must be verified to make sure it is complete, feasible, and consistent with the overall project. The team will verify it using the following techniques:

- **Team member review:** Every section is reviewed by a team member who did not write it before its pull request is merged. Before each submission, team members do a final full-document review according to their verification roles (Section 3.1), using the checklist below.
- **Peer review by classmates:** The assigned peer review team reviews the plan and reports issues on GitHub. Each issue is resolved or answered with a justification before the next revision.
- **Supervisor review:** In meetings with the supervising team, each supervisor reviews the sections related to their expertise. Dr. Yao Du reviews the speech-language therapy aspects of the plan, such as the transcription and report generation tests. Dr. Yang Yue reviews the security and code testing.

- **TA review:** The team's TA, Tanya Djavaherpour, answers questions about the VnV plan during development and gives feedback after submission.
- **Pilot testing:** Since TeLaC builds on a previous capstone project, the team will run a sample of system tests on the existing system before implementation is complete. The pilot will include tests for features that already exist, each run by a team member who did not write the test. Tests that cannot be followed or whose expected outputs prove unrealistic are revised, and the findings are logged as GitHub issues.

The following checklist will be used by the team:

- ❑ Does every functional requirement in the SRS have at least one system test?
- ❑ Does every nonfunctional requirement in the SRS have at least one system test?
- ❑ Is every requirement traced to its tests in the traceability table?
- ❑ Have all peer review and supervisor issues been addressed and closed on GitHub?
- ❑ Has every team member reviewed and approved at least one section they did not write?
- ❑ Can a team member who did not write a test follow its steps as written?
- ❑ Are all symbolic parameters used in the tests defined in the appendix?
- ❑ Have the findings from pilot testing been logged and the affected tests revised?
- ❑ Does each verification activity have a documentation standard and a deadline?

3.5 Implementation Verification

The implementation will be verified through the system tests in Section 4 and the unit tests in Section 5, and statically through the automated tools in Section 3.6. This section covers the manual checks that those tools cannot perform and that the team must do.

Every pull request is inspected by at least one other team member before it is merged (Development Plan). In addition to the general items in the pull request template, the reviewer checks that each endpoint verifies the clinician is allowed to access the requested patient (SR7), that no patient data is written to logs, and that failures from external services such as the transcription model or storage are handled without losing the clinician's work. The system tests (e.g., NFR-ST-RE2 and NFR-ST-CV3) check that work is kept under normal conditions, but not when an external service fails.

Finally, after the Revision 0 demo, the team will lead a code walkthrough with Dr. Yue, one of the project's supervisors and a software engineering professor at California State University San Marcos. The walkthrough will focus on the modules from the Hazard Analysis that handle patient data and transcripts, which are the account system, the database, and the transcription pipeline. During the walkthrough, the team will also look for race conditions, data handling problems, and security issues. Race conditions in particular are not covered by the system tests: for example, NFR-ST-CV2 only checks edits made on one device at a time, not two sessions saving edits to the same transcript chunk at once. The team will send him the relevant code by sharing a link to the GitHub repository, as well as a short list of questions beforehand. Each finding from the code walkthrough is to be recorded as a GitHub issue. Where possible, each issue is closed together with a new test that reproduces the problem, so that the fixes are documented and visible in case it ever happens again.

3.6 Automated Testing and Verification Tools

The following lists the automated testing and verification tools used throughout the software's lifecycle:

- Unit Tests: pytest (Python), Jest (TypeScript/JavaScript)
- Linters: Ruff (Python), ESLint (TypeScript/JavaScript)
- Continuous Integration: GitHub Actions

The CI workflows and tooling are described further in the Development Plan.

For code coverage, Coverage.py is used for the Python code. For TypeScript/JavaScript, Istanbul is used. This is considering it is built into Jest, which reduces the number of additional frameworks needed. Both tools report which lines and functions are not covered by the tests, so the team can see where additional tests are needed. The team's target is approximately 80% for line coverage, considering that is the current standard. Furthermore, it keeps the testing effort balanced with the time available for developing the software.

Ruff is used to ensure the Python coding standard (PEP 8) is respected. It was chosen for its speed compared to alternative linters. Additionally, since TypeScript/JavaScript makes up a large part of the codebase, ESLint is used to find and report problematic patterns and bugs in that code. ESLint is the industry standard for static analysis of JavaScript, which is why it was chosen. Moreover, GitHub Actions runs the unit tests and the linting checks on every pull request that is opened or updated and on every merge to main. A pull request with failing checks must be fixed before it can be merged.

3.7 Software Validation

To confirm that TeLaC meets the needs of its users and is the correct system to carry out the goals of speech-language pathologists, the team will validate the software through the following activities:

- **Requirements review sessions:** During one of the team's bi-weekly meetings, the team will meet with the research team to walk through each use case in the SRS, and note any discrepancies or missing requirements between what the clinician needs or expects in a step. All changes will be logged as GitHub issues and applied to the SRS, as described in the SRS Verification section.
- **Rev 0 demo with the supervisor:** Shortly after the scheduled Rev 0 demo, the team will demonstrate the system to the supervisor. The demo will follow the main workflow, from uploading an assessment recording to reviewing and editing the transcript, running the SAFA analysis, and viewing the generated report. All feedback will be logged as GitHub issues and used to revise the requirements and the implementation.

- **Task-based user testing:** Clinicians will be asked to complete realistic tasks, such as reviewing and annotating an assessment, without help from the team. The team will record task completion, time taken, and any points of confusion. Since one of the goals of the project is to reduce annotation time compared to manual transcription, the time taken will be compared against the current manual process. Participants will also complete a short questionnaire, and the results will be documented in the Usability Report.

- **Interviews with clinicians/researchers:** After user testing, the team will hold short one-on-one interviews with participating clinicians in connection with the research team. Questions will focus on how well the system fits into their current assessment workflow, whether they're able to trust the generated transcripts, SAFA, and clinical report results, and what features may be missing or unnecessary.

- **Sample assessment data:** The team will utilize sample assessment recordings from the research team to use for testing and demo purposes. Real recordings will show whether the transcription and SAFA analysis can actually produce results that clinicians find useful, as test data alone can't confirm.

4 System Tests

This section describes the system tests that will be used to check TeLaC against the requirements in the SRS. Tests are grouped by functional and nonfunctional requirements, further separated into specific theme sections. Each test states the requirement it covers so that results can be accurately traced back to the SRS, and is further mapped within a traceability matrix, ensuring that all expectations are met.

4.1 Tests for Functional Requirements

This section contains the tests for the functional requirements in the SRS, grouped into areas based on the part of the clinician's workflow they cover.

4.1.1 Data Collection and Storage

This section verifies that annotations are saved to the patient's assessment record, can be exported, and that recordings can be marked as reviewed.

1. FR-ST-DCS1: **Store Completed Annotations in the Assessment Record**

   Purpose: Verifies that the system stores a clinician's saved transcript annotations in the patient's assessment record.

   Type: Manual, Dynamic

   Initial State: A clinician account is logged in, a patient record exists, and the patient's assessment video is open in the annotation interface with its auto-generated transcript.

   Input: 3 assessment videos. For each video, 3 edits will be made to the transcript and documented with their corresponding chunks for later reference.

   Output: After saving, every edit appears in that patient's assessment record when opened, and still appears after logging out and back in.

   Test Case Derivation: FR19 requires that a saved annotation appear in the patient's assessment record from any clinician view without any file upload or download. This keeps patient information accurate and up to date and shortens the speech analysis workflow.

   How the test will be performed:

   1. In the annotation interface, make the listed edits to the transcript and save.
   2. Open the assessment from the patient record and check that each edit is shown.
   3. Log out, log back in, and check that the edits are still shown.

2. FR-ST-DCS2: **Export Annotation Data**

Purpose: Verifies that the system exports a video assessment's approved annotations in each supported file format, and that the exported file matches the stored annotations.

Type: Manual, Dynamic

Initial State: A clinician account is logged in, a patient record exists, and the patient's assessment video is open in the annotation interface with its auto-generated transcript.

Input: 3 assessment videos. For each video, 3 edits will be made to the transcript and documented with their corresponding chunks for later reference.

Output: For each supported file format, a file is exported that contains every approved annotation for that assessment, with the same content as the edits made in the annotation interface.

Test Case Derivation: FR3 requires that clinicians can export annotation data from an assessment in a supported file format. Additionally, the file must contain all approved annotations for that assessment. This lets clinicians record, review, and share results without relying on other software.

How the test will be performed:

1. Make and approve the listed edits to the transcript in the annotation interface.
2. Export the annotations in one supported file format.
3. Open the exported file and check that every edit appears in it with the correct content.
4. Repeat steps 2 - 3 for each remaining supported file format.

3. FR-ST-DCS3: **Return Annotated Results to the Patient Record**

Purpose: Verifies that when a clinician finishes annotating a transcript, the system shows the number of verified and unverified segments, saves the annotated transcript to the patient's assessment record, and shows it with the assessment results.

Type: Manual, Dynamic

Initial State: A clinician account is logged in, a patient record exists, and the patient's assessment video is open in the annotation interface with its auto-generated transcript.

Input: 2 assessment videos where assessment 1 has all segments verified and assessment 2 has only 10 segments verified and the remaining segments are left unverified.

Output: Before confirming, the system shows how many segments are verified and how many are not. After the clinician confirms, the system returns to the assessment's results and correctly shows the annotated transcript with the results and notes. For assessment 1, the transcript is marked as verified. For assessment 2, the transcript is saved and the unverified segments are indicated.

Test Case Derivation: UC10 requires that a finished annotation is saved to the patient's assessment record and shown with the assessment, and that unverified segments are indicated when annotation is completed without verifying everything. This lets clinicians keep track of which transcripts they have reviewed.

How the test will be performed:

1. For each video, verify the listed segments in the annotation interface and indicate that the annotation is finished.
2. Check that the counts of verified and unverified segments shown match what was verified, then confirm that the annotation is complete.
3. Check that the system returns to the assessment's results and shows the annotated transcript with the results and notes.
4. Check that assessment 1's transcript is marked verified, and that assessment 2's unverified segments are indicated.

4. FR-ST-DCS4: **Mark Patient Recording as Reviewed**

Purpose: Verifies that a clinician can mark a recording as reviewed, the review status is updated and shown, and that it per-

sists when the recording is reopened.

Type: Manual, Dynamic

Initial State: A clinician account is logged in, a patient record exists, the patient's video assessment is open, and its recording has not been marked as reviewed.

Input: A patient assessment recording that has not been marked as reviewed, including 2 notes made on separate questions while reviewing.

Output: Once the recording is marked as reviewed, the system shows the updated review status. When the assessment is reopened later, it is still shown as reviewed and the notes are unchanged.

Test Case Derivation: UC14 requires that the system updates and displays a recording's review status once the clinician marks it as reviewed. This helps clinicians tell which recordings they have finished reviewing and which are ready to be processed further.

How the test will be performed:

1. Open the recording in the Individual Assessment View, watch it, and make 2 notes.
2. Select the option to mark the recording as reviewed.
3. Check that the recording is now shown as reviewed.
4. Leave the Individual Assessment View, then reopen the same recording and check that it is still shown as reviewed and the notes have not changed.

4.1.2 Video and Audio Analysis

This section verifies that assessment videos are transcribed, speakers can be told apart, and interference and language errors are flagged.

1. FR-ST-VAA1: **Auto-Generate Transcript for Video Assessments**

Purpose: Verifies that the system automatically generates a transcript for assessment videos without clinician intervention.

Type: Manual, Dynamic

Initial State: A clinician account is logged in, a patient record exists, and all assessment videos are in the system.

Input: 10 assessment videos.

Output: For each video, a non-empty transcript is generated and available for the clinician to review. At least GENERATE_SUCCESS% of the videos must produce a transcript (NFR17).

Test Case Derivation: FR1 requires that a transcript is generated for every supported video without any action from the clinician, and (NFR17) sets the required success rate. Additionally, a working auto-generation transcript tool is critical to streamlining the SLP workflow. **How the test will be performed:**

1. For each assessment video, open it in the annotation tool, and generate a transcription
2. Record whether each video's transcript was generated and appears in the annotation interface
3. Compute the success rate and compare it to GENERATE_SUCCESS%.

2. FR-ST-VAA2: **Distinguish Speakers in Transcripts**

Purpose: Verifies that the system visually distinguishes between speakers in a transcript, so the clinician can tell which speaker said each utterance.

Type: Manual, Dynamic

Initial State: A clinician is logged in, a patient record exists, and a transcript has already been generated for each assessment video.

Input: 10 assessment videos each containing at least two speakers. For each video, an answer key will be created by the team prior

to testing by watching the video assessment. The answer key will contain the true speakers at 5 different timestamps.

Output: In every auto-generated transcript, each identified speaker is shown with a distinct label, tag, or highlight. For each of the 5 timestamps in the answer key, the label displayed by the system on that utterance must correspond to that on the answer key.

Test Case Derivation: FR4 requires that each speaker be distinguishable and that the clinician can tell which speaker corresponds to each utterance. Clinicians use this to annotate the right individual and account for outside interferences such as a parent prompting the child. **How the test will be performed:**

1. For each assessment video, open it in the annotation tool, and generate a transcription
2. Record whether each video's transcript was generated and appears in the annotation interface
3. Compute the success rate and compare it to GENERATE_SUCCESS%.

3. FR-ST-VAA3: **Flag Potential Interference in Assessments**

Purpose: Verifies that the system flags questions where potential interference may have occurred and marks them visibly in the Individual Assessment View.

Type: Manual, Dynamic

Initial State: A clinician account is logged in, a patient record exists, and the assessment video is already in the system.

Input: A video assessment containing 3 moments of interference (e.g., a guardian giving a hint during a question). A list of the question numbers where interference occurs is created by watching the video before testing.

Output: In the Individual Assessment View, every question on the list is flagged by the system.

Test Case Derivation: FR13 requires that each question with potential interference is flagged. Interference can make results invalid, so clinicians need to see which questions are affected.

How the test will be performed:

1. Open the Individual Assessment View for the assessment.
2. For each question number on the list, check that the question is flagged by the system.
3. If all listed questions are flagged, the test passes.

4. FR-ST-VAA4: **Show Grammar and Phonology Errors as Suggestions**

Purpose: Verifies that the system identifies potential grammatical and phonological errors in a transcript and presents them to the clinician as unverified suggestions.

Type: Manual, Dynamic

Initial State: A clinician account is logged in, a patient record exists, and a transcript has been generated for each assessment video.

Input: 5 assessment videos in English and/or Mandarin, and each containing known grammatical and phonological errors. For each video, an answer key lists the timestamp and type (grammar or phonology) of each known error, created by the team before testing by watching the video.

Output: Each known error in the answer key is shown to the clinician as a suggestion linked to its transcript segment and marked as unverified. An error appears in the assessment report only after the clinician accepts it, and not if it is rejected or left unreviewed.

Test Case Derivation: FR26 requires that detected errors are linked to their transcript segment and shown as unverified until the clinician accepts or rejects them. Ambiguous speech can be clinically meaningful, so the system may only suggest errors and the clinician must make the final decision.

How the test will be performed:

1. Open each video's transcript in the annotation interface.
2. For each error in the answer key, check that the system shows it as a suggestion on the matching transcript segment, marked as unverified.
3. Accept some suggestions and reject others, then generate the assessment report and check that only the accepted errors appear in it.

4.1.3 Authentication and Accounts

This section verifies that clinicians can create an account, log in, and log out, and that invalid attempts are rejected.

1. FR-ST-A1: **Create Account with a New Email**

Purpose: Verifies that a clinician can create an account with an email that is not already registered to a verified clinician account.

Type: Manual, Dynamic

Initial State: The system is running, no user is logged in, and the email address is not registered and stored in the account database.

Input: A name, an email address the tester can access, and a password of at least MIN_PASSWORD_LENGTH characters.

Output: The account is created and the system tells the clinician to verify their email.

Test Case Derivation: UC12 requires that a clinician can create an account so they can access the system.

How the test will be performed:

1. Open the account creation page.
2. Enter all the input details and submit.
3. Check that the account is created and the email verification message is shown.

2. FR-ST-A2: **Create Account with an Existing Email**

Purpose: Verifies that the system rejects account creation with an email that is already registered to a clinician account.

Type: Manual, Dynamic

Initial State: The system is running, no user is logged in, and a clinician account already exists for the email address.

Input: A name, the already registered email address, and a valid password.

Output: No new account is created. The system shows a message stating that the email is already registered and prompts the clinician to log in instead.

Test Case Derivation: UC12 requires that duplicate accounts are rejected, which keeps each patient record tied to one clinician account.

How the test will be performed:

1. Open the account creation page.
2. Enter all the input details and submit.
3. Check that the account is rejected with the correct message stating that the email is already in use, and that only one account exists for the email address.

3. FR-ST-A3: **Require Email Verification Before Use**

Purpose: Verifies that a new account cannot be used until its email address has been verified.

Type: Manual, Dynamic

Initial State: A clinician account has been created, but its email address has not been verified.

Input: The account's email address and password, entered once before and once after following the verification link sent to the account's inbox.

Output: Before verification, login is blocked and the system shows a message stating that the email address must be verified first. After verification, login succeeds.

Test Case Derivation: UC12 requires email verification so that only real clinician email addresses can access patient data.

How the test will be performed:

1. Try to log in with the unverified account and check that access is blocked with the correct message.
2. Open the verification email and follow the link.
3. Log in again and check that it succeeds.

4. FR-ST-A4: **Log In with Valid Credentials**

Purpose: Verifies that a clinician with a verified account can log in.

Type: Manual, Dynamic

Initial State: A verified clinician account exists, and no user is logged in.

Input: The account's correct email address and password.

Output: The clinician is logged in and taken to their patient dashboard.

Test Case Derivation: UC1 requires that clinicians with valid credentials can access the system.

How the test will be performed:

1. Open the login page.
2. Enter the correct credentials and submit.
3. Check that the clinician's patient dashboard is shown.

5. FR-ST-A5: **Log In with Invalid Credentials**

Purpose: Verifies that login is denied when the credentials are incorrect.

Type: Manual, Dynamic

Initial State: A verified clinician account exists, and no user is logged in.

Input: Two login attempts documented by the team prior to testing: the registered email address with an incorrect password, and an email address not registered to any account.

password, and an unregistered email address with any password.

Output: Both attempts are denied with the same error message and no patient data is shown. The message does not reveal whether the email address is registered.

Test Case Derivation: UC1 requires that invalid login attempts are rejected, which protects patient data from unauthorized access.

How the test will be performed:

1. Enter the first set of credentials, submit, and check that login is denied with an error message.
2. Repeat step 1 for the second set of credentials.
3. Check that both error messages are the same.

6. FR-ST-A6: **Log Out**

Purpose: Verifies that a clinician can log out and that patient data cannot be accessed afterwards.

Type: Manual, Dynamic

Initial State: A clinician is logged in and viewing their patient dashboard.

Input: Selecting the log out option.

Output: The clinician is returned to the login page, and pressing the browser's back button does not show any patient data.

Test Case Derivation: UC2 requires that clinicians can end their

session so that others using the same device cannot access patient data.

How the test will be performed:

1. Select the log out option and check that the login page is shown.
2. Press the browser's back button and check that no patient data is shown.

4.1.4 Patient Management

This section verifies that clinicians see only their own patients, can add patients, and can browse each patient's assessments.

1. FR-ST-PM1: **Show Only Assigned Patients**

Purpose: Verifies that the patient list shows exactly the patients assigned to the logged-in clinician.

Type: Automatic, Dynamic

Initial State: The test database contains Clinician A with 3 assigned patients and Clinician B with 2 different patients.

Input: A request for the patient list as Clinician A.

Output: The list contains exactly Clinician A's 3 patients and none of Clinician B's.

Test Case Derivation: FR9 requires that clinicians can view their assigned patients. Showing other clinicians' patients would expose protected health information.

How the test will be performed:

1. The automated test (pytest) loads seed data created by the team prior to testing, containing Clinician A with 3 assigned patients and Clinician B with 2 different patients.
2. The test logs in as Clinician A and requests the patient list.

3. The test checks that the returned list matches Clinician A's patients exactly.

2. FR-ST-PM2: **Show an Empty List with No Assigned Patients**

Purpose: Verifies that a clinician with no assigned patients sees an empty list rather than an error or other clinicians' patients.

Type: Automatic, Dynamic

Initial State: The test database contains Clinician C with no assigned patients and Clinician A with 3 patients.

Input: A request for the patient list as Clinician C.

Output: An empty list is returned without errors.

Test Case Derivation: FR9 requires that the patient list reflects the clinician's assignments, including when there are none.

How the test will be performed:

1. The automated test (pytest) loads seed data created by the team prior to testing, containing Clinician C with no assigned patients and Clinician A with 3 patients.
2. The test logs in as Clinician C and requests the patient list.
3. The test checks that the list is empty and no error is returned.

3. FR-ST-PM3: **Add a New Patient**

Purpose: Verifies that a clinician can add a new patient, who then appears in their patient list.

Type: Automatic, Dynamic

Initial State: Clinician A is logged in with 3 assigned patients.

Input: A new patient record with a name, a date of birth, and Mandarin and English as the languages spoken.

Output: The patient list contains 4 patients, including the new patient with the entered details.

Test Case Derivation: FR10 and UC3 require that clinicians can add patients so their assessments can be recorded.

How the test will be performed:

1. The automated test (pytest) loads seed data created by the team prior to testing, containing Clinician A with 3 assigned patients, then submits the new patient record as Clinician A.
2. The test requests the patient list.
3. The test checks that the new patient appears with the correct details and that the list has 4 patients.

4. FR-ST-PM4: **Show Assessments Newest to Oldest**

Purpose: Verifies that a patient's completed assessments are listed from newest to oldest.

Type: Automatic, Dynamic

Initial State: A patient assigned to Clinician A has 3 completed assessments with 3 different dates, documented by the team prior to testing for later reference.

Input: A request for that patient's assessments as Clinician A.

Output: All 3 assessments are returned, ordered from newest to oldest.

Test Case Derivation: FR11 and UC13 require that clinicians can browse a patient's assessments. Newest-first ordering lets clinicians quickly find the most recent results.

How the test will be performed:

1. The automated test (pytest) loads seed data created by the team prior to testing, containing the patient's 3 assessments with their documented dates.
2. The test requests the patient's assessment list.
3. The test checks that all 3 are returned in descending date order.

4.1.5 Transcript Review and Editing

This section verifies that clinicians can correct the automatically generated transcript, assign a role to each speaker, filter the transcript by role, and choose which parts of an assessment to open in the SAFA tool. All tests in this section and in the Playback and Navigation, Performance, and Convenience sections use test recordings, since real patient data is not to be used during development. Test assessment A is an approximately 5-minute recording involving two speakers (child and parent), speech in more than one supported language (English and Mandarin), 10 questions, and about 15 transcript segments. Test assessment B is a recording of the maximum length allowed by the system (MAX_REC_LEN minutes) that will be used for timing tests. For each assessment, the stored transcript (each segment's start and end time, speaker, and the time each question was asked) is recorded before testing and serves as the answer key.

1. FR-ST-TR1: **Edit and Save Transcript Text**

Purpose: Verifies that a clinician can add, delete, and modify transcript text. Also verifies that saved changes persist, and that unsaved changes can be discarded.

Type: Manual, Dynamic

Initial State: A clinician account is logged in, and test assessment A is open in the SAFA tool with its auto-generated transcript, with no segments edited.

Input: 5 segments of test assessment A, documented before testing with their original text. In segment 1 a word is added, in segment 2 a word is deleted, and in segment 3 a word is replaced and its language tag is changed. Segment 4 is edited and then discarded, and segment 5 is edited and left unsaved.

Output: Segments 1–3 show the edited text after saving, their original auto-generated text is kept and the differences are shown, and each is marked as verified with who made the change and when. Segment 4 returns to its original text. Trying to leave the transcript with segment 5 unsaved prompts the clinician to save or discard. After closing and reopening the assessment, segments

1–3 still show the edited text.

Test Case Derivation: FR2 requires that clinicians can add, delete, and modify transcript text and that saved changes persist when the recording is reopened. UC9 adds that the original version is kept, the segment is marked as verified, and unsaved changes can be discarded or prompt the user before leaving. Clinicians rely on these corrections being kept, since all later annotations and analysis are based on the corrected text.

How the test will be performed:

1. Make the listed edits to segments 1–3 and save each one.
2. Check that each edited segment shows the new text, the original text, the differences, and its verified status with the editor and time.
3. Edit segment 4, discard the change, and check that it shows its original text.
4. Edit segment 5 without saving, try to leave the transcript, and check that the system asks whether to save or discard. Discard the change.
5. Close the assessment, reopen it, and check that segments 1–3 still show the edited text.

2. FR-ST-TR2: **Assign a Role to Each Detected Speaker**

Purpose: Verifies that once a clinician assigns a role to a speaker the system detected, every segment from that speaker is labeled with the role.

Type: Manual, Dynamic

Initial State: A clinician account is logged in, and test assessment A is open in the SAFA tool with its auto-generated transcript and two detected speakers, neither of which have a role.

Input: The roles Child and Parent are each assigned once to the matching detected speaker. One of the segments contains both speakers.

Output: Every segment from each speaker shows that speaker's role, and the number of segments labeled with each role matches the answer key. In the segment with both speakers, each speaker's part is labeled with their own role. The roles are still shown after the assessment is closed and reopened.

Test Case Derivation: FR6 requires that every segment from a detected speaker is labeled with the role the clinician assigns, and UC5 requires that each speaker in a segment with more than one speaker is labeled separately. Clinicians need the child's speech separated from the adults' to analyze the child's language and to find parent interference.

How the test will be performed:

1. Select a segment from the first speaker and assign the role Child.
2. Repeat step 1 for the role Parent.
3. Count the segments labeled with each role and compare the counts to the answer key.
4. Check that each speaker in the segment with both speakers is labeled with their own role.
5. Close and reopen the assessment, and check that all roles are still shown.

3. FR-ST-TR3: **Filter the Transcript by Speaker Role**

Purpose: Verifies that filtering the transcript by a speaker role shows exactly that speaker's segments.

Type: Manual, Dynamic

Initial State: A clinician account is logged in, and test assessment A is open in the SAFA tool with the roles from FR-ST-TR2 assigned.

Input: The filters Child, then Parent, then no filter.

Output: For Child and Parent, exactly the segments the answer

key lists for that role are shown, and no others. The segment containing both speakers is shown under both filters. With no filter, all segments are shown again in their original order.

Test Case Derivation: FR6 requires that clinicians can filter the transcript by speaker role. Clinicians use this to review only the child's answers, or only the parent's speech when checking for interference, so a filter that hides or adds segments would lead to an incomplete analysis.

How the test will be performed:

1. Filter by Child and compare the segments shown to the answer key's Child segments.
2. Repeat step 1 for Parent.
3. Remove the filter and check that all segments are shown in their original order.

4. FR-ST-TR4: **Open Selected Questions in the SAFA Tool**

Purpose: Verifies that a clinician can choose which questions, or the full assessment, to open in the SAFA tool, and that only those portions are opened.

Type: Manual, Dynamic

Initial State: A clinician account is logged in, and the assessment overview of test assessment A is open.

Input: Questions 3 and 7 are selected and opened in the SAFA tool. Then the full assessment is selected and opened.

Output: When questions 3 and 7 are opened, the SAFA tool shows only the segments and the portions of the recording for those 2 questions, and playback does not continue into other questions. When the full assessment is opened, all segments and the full recording are shown.

Test Case Derivation: FR15 requires that the clinician can select one or more questions, or the full assessment, and that only those portions are opened in the SAFA tool. This supports the

On-Demand Refinement workflow, where clinicians only annotate flagged or unclear questions to save time.

How the test will be performed:

1. From the overview, select questions 3 and 7 and open them in the SAFA tool.
2. Compare the segments shown with the answer key's segments for questions 3 and 7.
3. Play the recording to the end of question 3 and check that playback does not continue into question 4.
4. Return to the overview, select the full assessment, open it, and check that all segments are shown.

4.1.6 Playback and Navigation

This section verifies that the transcript and recording stay linked, so that clinicians can follow along during playback and jump to any segment or question, and that clinicians can move between workflow stages without losing saved work. The timing tolerances for these features are tested separately in the Performance section.

1. FR-ST-PN1: **Highlight the Current Segment During Playback**

Purpose: Verifies that during playback the system highlights the segment being spoken and scrolls the transcript so that it is visible.

Type: Manual, Dynamic

Initial State: A clinician account is logged in, and test assessment A is open in the SAFA tool with playback at the start of the recording.

Input: 10 playback times chosen from the answer key, spread across the recording. One is in a segment that is off-screen when playback starts, and one is reached by dragging the playback bar instead of playing.

Output: At each of the 10 times, the highlighted segment is the one whose start and end times contain the playback position, and

it is visible in the transcript without the clinician scrolling.

Test Case Derivation: FR20 requires that the highlighted segment is the one whose start and end times contain the playback position, and that the transcript scrolls to keep it visible. UC8 describes clinicians following the transcript while watching the recording, which only works if the highlight is on the right segment.

How the test will be performed:

1. Play the recording and pause at the first listed time.
2. Check that the highlighted segment's start and end times, from the answer key, contain the playback position, and that the segment is visible.
3. Repeat steps 1–2 for the remaining times. For the time listed as reached by dragging, drag the playback bar to it instead of playing.

2. FR-ST-PN2: **Jump to a Selected Segment**

Purpose: Verifies that selecting a transcript segment moves playback to the start of that segment and highlights it.

Type: Manual, Dynamic

Initial State: A clinician account is logged in, and test assessment A is open in the SAFA tool.

Input: 5 segments: the first, the last, one in the middle, one that is off-screen, and one shorter than 1 second. Each is selected once while playback is paused and once while it is playing (10 selections in total).

Output: After each selection, the playback position is at the segment's start time from the answer key, and that segment is highlighted.

Test Case Derivation: FR21 requires that selecting a segment sets the playback position to its start time and highlights it. Clinicians use this to re-watch a child's answer or a flagged moment,

and the first, last, and very short segments are the ones most likely to be handled wrongly.

How the test will be performed:

1. With playback paused, select the first listed segment.
2. Check that the playback position matches the segment's start time and the segment is highlighted.
3. Repeat steps 1–2 for the remaining segments.
4. Repeat steps 1–3 while the recording is playing.

3. FR-ST-PN3: **Jump to a Selected Question or Moment**

Purpose: Verifies that selecting a question or a flagged moment starts playback at the point in the recording where it occurred.

Type: Manual, Dynamic

Initial State: A clinician account is logged in, and test assessment A is open in the Individual Assessment View with at least one flagged moment.

Input: 5 questions selected from the question list, including the first and last question, and 1 flagged moment.

Output: After each selection, playback starts at the time the answer key lists for when that question was asked or that moment occurred.

Test Case Derivation: FR14 requires that selecting a question or a specific moment starts playback at that point in the assessment. This saves clinicians from searching through the whole recording to find a question.

How the test will be performed:

1. Select the first listed question.
2. Check that playback starts at the time listed in the answer key for that question.

3. Repeat steps 1–2 for the remaining questions and the flagged moment.

4. FR-ST-PN4: **Move Between Workflow Stages Without Losing Saved Work**

Purpose: Verifies that clinicians can move forward and back between all workflow stages without losing saved work, and that the system shows the current and completed stages.

Type: Manual, Dynamic

Initial State: A clinician account is logged in, and test assessment A is open in the assessment overview, with no saved work.

Input: The clinician moves from the overview to the SAFA tool and saves 2 transcript edits, then to the evaluation stage and saves a grade for 1 question, then to the report stage. The clinician then goes back to each earlier stage in reverse order.

Output: At every stage, both edits and the grade are still shown. The progress indicator always shows the current stage and marks each stage already visited as completed.

Test Case Derivation: FR5 requires that clinicians can move between all workflow stages without losing saved work, and that the system shows the current and completed stages. Clinicians do not analyze an assessment in a straight line, and often go back to correct earlier work.

How the test will be performed:

1. Move through the stages in order, saving the listed edits and grade, and check the progress indicator at each stage.
2. Go back to each earlier stage in reverse order.
3. At each stage, check that both edits and the grade are shown and the progress indicator is correct.

4.1.7 Assessment Evaluation and Notes

This section verifies that clinicians can manually grade assessment questions, create notes on individual questions and assessments as a whole, and update results at any time.

1. FR-ST-AE1: **Manually Grade a Question During Playback**

Purpose: Verifies that clinicians can manually evaluate questions that cannot be graded automatically while playing back the part of the recording where the question was answered.

Type: Manual, Dynamic

Initial State: Clinician is logged into their account, a patient record exists, and a completed assessment containing questions that aren't automatically graded is open in the Individual Assessment View.

Input: 1 assessment video containing 3 repetition questions that are not automatically graded. The clinician marks 2 of them as correct and 1 as incorrect.

Output: For each of the 3 questions, the recording segment for that question plays while the clinician records an evaluation. After saving, each evaluation is shown with its question, and the assessment's score updates to include the 3 evaluations.

Test Case Derivation: FR16 requires that the clinician can play the recording segment for every question that is not automatically graded and record an evaluation, which must then be shown with the question and included in the assessment's score. Some questions may have no single correct answer, so the clinician must be able to judge them from the recording.

How the test will be performed:

1. Record the assessment's score before grading.
2. For each of the 3 questions, select one of them and check that the matching segment of the recording plays, and then record the evaluation.

3. Save, then check that each evaluation is shown with its question and that the score has changed to include the 3 evaluations.

2. FR-ST-AE2: **Add, Edit, and Delete a Question Note**

Purpose: Verifies that clinicians can add, edit, and delete a note on an individual assessment question, and that any notes saved stay attached to that question.

Type: Manual, Dynamic

Initial State: Clinician is logged into their account, a patient record exists, and a completed assessment is open in the Individual Assessment View.

Input: A note with the text "Substituted /s/ with /th/" is added to question 3 of the assessment, then edited to "Substituted /s/ with /th/ twice", then the note is deleted.

Output: After the note is added, it is shown with question 3 and with no other question, including after the assessment is closed and reopened. After it is edited, only the edited text is shown. After it is deleted, no note is shown on question 3 when the assessment is reopened.

Test Case Derivation: FR17 requires that a note saved on a question is shown with that question each time, unless the clinician edits or deletes it, as clinicians use these notes to record observations that support their final evaluation.

How the test will be performed:

1. Add a note to question 3, save. Close and reopen the assessment, and check that the note is shown on question 3 only.
2. Edit the note, save. Close and reopen the assessment, and check that only the edited text is shown.
3. Delete the note. Close and reopen the assessment, and check that question 3 has no note.

3. FR-ST-AE3: **Add, Edit, and Delete an Assessment Note**

Purpose: Verifies that clinicians can add, edit, and delete notes for an entire video assessment.

Type: Manual, Dynamic

Initial State: Clinician is logged into their account, a patient record exists, and a completed assessment is open in the Individual Assessment View.

Input: A note on the whole assessment with the text "Child was distracted by background noise in the second half", then edited to "Child was distracted by background noise after question 10", then deleted.

Output: After the note is added, it is shown on the assessment's view, including after the assessment is closed and reopened. After it is edited, only the edited text is shown. After it is deleted, no assessment note is shown when the assessment is reopened.

Test Case Derivation: FR18 requires that a note saved on an assessment is shown on the assessment's view every time it is reopened, unless the clinician edits or deletes it. For example, some observations, such as the child's attention or the recording conditions, can apply to the whole session rather than just a single question.

How the test will be performed:

1. Add the note to the assessment, save. Close and reopen the assessment, and check that the note is shown.
2. Edit the note, save. Close and reopen the assessment, and check that only the edited text is shown.
3. Delete the note. Close and reopen the assessment, and check that no assessment note is shown.

4. FR-ST-AE4: **Edit a Previously Saved Result**

Purpose: Verifies that clinicians can change an assessment result after it has been saved, and that the updated result is displayed.

Type: Manual, Dynamic

Initial State: Clinician is logged into their account, a patient record exists, and a completed assessment has a saved result where question 1 is marked as correct.

Input: The clinician changes the result of question 1 from correct to incorrect.

Output: Once the change is submitted, question 1 is shown as incorrect and the assessment's overall score decreases by one question. The previous value is no longer shown.

Test Case Derivation: FR8 requires that the system displays the updated result once the clinician submits a change. Clinicians may notice an error or change their judgment after reviewing more of the recording, so they need to update results at any stage.

How the test will be performed:

1. Record the result of question 1 and the overall score.
2. Change question 1 from correct to incorrect and submit.
3. Check that question 1 is shown as incorrect and that the overall score has decreased by one question.

5. FR-ST-AE5: **Save Results After Leaving the Assessment**

Purpose: Verifies that grades and result changes are saved when the clinician leaves an assessment and returns to it later.

Type: Manual, Dynamic

Initial State: Clinician is logged into their account, a patient record exists, and a completed assessment is open in the Individual Assessment View.

Input: The clinician grades 2 questions that are not automatically graded and changes the result of 1 automatically graded question, then leaves the assessment in 2 ways: by opening another patient's

assessment, and by logging out.

Output: Each time the clinician returns to the assessment, all 3 results are shown exactly as they were saved, and the overall score is the same as before leaving.

Test Case Derivation: FR8 requires that clinicians can edit and update results at any time, which is only possible if earlier changes are kept between visits. Clinicians do not always analyze assessments in one sitting, so their work must still be there when they come back.

How the test will be performed:

1. Make the 3 result changes, save, and record the overall score.
2. Open another patient's assessment, then return to the original one, and check that the 3 results and the score are unchanged.
3. Log out, log back in, open the assessment, and check again.

4.1.8 Results Display and Reporting

This section verifies that assessment results are shown in a designated view, organized by clinical domain and language, displayed in a format suited to the assessment's purpose, and included in reports with the clinician's edits.

1. FR-ST-RD1: **Display the Assessment Results Overview**

Purpose: Verifies that the system shows an overview of an assessment's results, including flagged items and overall results, without requiring a transcript.

Type: Manual, Dynamic

Initial State: Clinician is logged into their account, a patient record exists, and the patient has completed assessments that have not been opened in the SAFA tool and have no transcript reviewed.

Input: 2 completed assessments, one with 2 questions flagged for interference and one with no flagged questions.

Output: For each assessment, the overview shows the patient's

overall results and each question's result (e.g., right or wrong). In the first assessment, the 2 flagged questions are marked as flagged; in the second, none are marked. The overview loads without the clinician opening the SAFA tool or a transcript.

Test Case Derivation: FR12 requires that the overview for each completed assessment shows the overall results, each question's result, and whether it was flagged, and that it can be viewed before the assessment is reviewed in the SAFA tool. Clinicians reviewing many assessments need a quick summary without in-depth annotation.

How the test will be performed:

1. From the patient's list of completed assessments, open the overview of each assessment.
2. Check that the overall results and each question's result are shown.
3. Check that exactly the 2 expected questions are flagged in the first assessment and none in the second.

2. FR-ST-RD2: **Organize Results by Clinical Domain**

Purpose: Verifies that every question is assigned to a clinical domain and that results can be viewed grouped and summarized by domain.

Type: Manual, Dynamic

Initial State: Clinician is logged into their account, a patient record exists, and a completed assessment is open.

Input: 1 completed assessment with questions covering semantics, grammar, and phonology, along with a list prepared by the team of each question's expected domain.

Output: Every question is assigned to the domain given in the list, and the results view groups the questions under semantics, grammar, and phonology, with a summary score for each domain.

Test Case Derivation: FR23 requires that every question in an

assessment is assigned to a domain and that the clinician can view results grouped and summarized by domain. Clinicians interpret a child's language abilities by domain, which supports diagnosis and reporting.

How the test will be performed:

1. Open the assessment's results grouped by domain.
2. Check that every question appears under the domain given in the list, and that no question is missing a domain.
3. Check that each domain's summary score matches the results of the questions listed under it.

3. FR-ST-RD3: **Visualize Results by Domain and Language**

Purpose: Verifies that results for an assessment completed in more than one language are displayed by clinical domain in a single consolidated view. This test only applies if the result visualization feature is implemented, since FR25 is linked to a stretch goal in the Problem Statement.

Type: Manual, Dynamic

Initial State: Clinician is logged into their account and a bilingual patient record exists.

Input: 1 assessment completed by the patient in both English and Mandarin, with results in semantics, grammar, and phonology for each language.

Output: A single view shows each domain's results for both languages side by side, so the clinician does not need to switch between views to compare the languages.

Test Case Derivation: FR25 requires that for an assessment completed in more than one language, the results for each clinical domain are displayed in a single consolidated view. Seeing both languages together helps clinicians tell a language difference from a disorder.

How the test will be performed:

1. Open the results visualization for the bilingual assessment.
2. Check that each of the 3 domains shows results for both English and Mandarin in the same view.
3. Check that the values shown match the results stored for each language.

4. FR-ST-RD4: **Generate a Report Containing Clinician Edits**

Purpose: Verifies that a generated assessment report includes every change the clinician made, and the automatically generated content wherever no change was made.

Type: Manual, Dynamic

Initial State: Clinician is logged into their account, a patient record exists, and a completed assessment has an automatically generated transcript.

Input: 1 assessment where the clinician makes 3 transcript edits, changes the result of 1 flagged question, and adds a note to 1 question. A list of these changes is recorded by the team before generating the report.

Output: The generated report contains all 5 changes from the list. Every part of the transcript and results that was not changed appears in the report as it was automatically generated.

Test Case Derivation: FR7 requires that the report includes every change the clinician made during analysis, and the automatically generated content where no change was made. The report must reflect the clinician's judgment, not only the automated output.

How the test will be performed:

1. Make the listed changes to the assessment and save.
2. Generate the assessment report.
3. Check that each of the 5 changes appears in the report.

4. Compare a sample of unchanged transcript segments and results in the report against the automatically generated content and check that they match.

5. FR-ST-RD5: **Display Results by Assessment Purpose**

Purpose: Verifies that the system displays assessment results in a format suited to the assessment's purpose.

Type: Manual, Dynamic

Initial State: Clinician is logged into their account and a patient record exists with completed assessments.

Input: 1 diagnostic assessment completed once, and 1 progress monitoring assessment completed 3 times on different dates with scores of 50%, 70%, and 85%.

Output: The diagnostic assessment's results are shown as a single session summary. The progress monitoring assessment's results are shown as change over time, with all 3 sessions listed in date order with their scores.

Test Case Derivation: FR22 requires that progress monitoring results are shown as change over time and diagnostic or screening results as a single session summary, since each purpose needs a different view to be interpreted correctly.

How the test will be performed:

1. Open the diagnostic assessment's results and check that only that session's summary is shown.
2. Open the progress monitoring assessment's results and check that all 3 sessions are shown in date order with scores of 50%, 70%, and 85%.

4.1.9 Assessment Interface

This section verifies the child-facing side of the assessment, where breaks keep children engaged during long assessments.

1. FR-ST-AI1: **Breaks During a Child Assessment**

Purpose: Verifies that the system shows a break activity after the configured questions and then resumes the assessment where it left off.

Type: Manual, Dynamic

Initial State: A test patient account is logged in on the patient-facing site, and 2 assessments are assigned to it.

Input: A 15-question assessment configured with breaks after questions 5 and 10, and a second assessment with no breaks configured. Each is completed from start to finish.

Output: In the first assessment, a break activity is shown after questions 5 and 10 only, and the assessment then continues from questions 6 and 11, with all earlier answers kept. In the second assessment, no break activity is shown.

Test Case Derivation: FR24 requires that a break activity is shown after the number of questions set for each assessment, and that the assessment then resumes where it left off. Children with limited attention may disengage during long assessments, but a break must not cause answers to be lost or questions to be skipped.

How the test will be performed:

1. Start the first assessment and answer each question.
2. Record after which questions a break activity is shown, and which question is shown after each break ends.
3. After finishing, check from the clinician side that all 15 answers were saved.
4. Complete the second assessment and check that no break activity is shown.

4.2 Tests for Nonfunctional Requirements

This section contains the tests for the nonfunctional requirements in the SRS, grouped by the same categories used in the SRS.

4.2.1 Usability

This section verifies that clinicians can use the system without training, find each function from its label, move between workflow stages easily, and see the result of each change right away.

1. NFR-ST-U1: **Complete Core Tasks Without Training**

> **Purpose:** Verifies that the system is intuitive for clinicians with limited technical fluency.
>
> **Type:** Nonfunctional, Dynamic, Manual
>
> **Initial State:** The system is running with a test clinician account for each participant, and each account has 1 test patient with 1 completed assessment. The participants have not used the system before and receive no training.
>
> **Input:** 5 clinicians each complete 3 core tasks without assistance: add a patient, assign a speaker role in an assessment transcript, and view an assessment's results.
>
> **Output:** For each core task, at least TASK_SUCCESS_RATE% of the clinicians complete the task without assistance within TASK_TIME minutes NFR1.
>
> **How the test will be performed:**
>
> 1. Give each clinician the list of tasks without explaining how to complete them.
> 2. Time each task from when the clinician starts until it is completed, and note any task the clinician gives up on or asks for help with.
> 3. For each task, calculate the percentage of clinicians who completed it without assistance within TASK_TIME minutes and compare it to TASK_SUCCESS_RATE%.

2. NFR-ST-U2: **Find Functions From Their Labels**

Purpose: Verifies that all functionality is presented with clear and descriptive labels.

Type: Nonfunctional, Dynamic, Manual

Initial State: The system is running with a test clinician account for each participant, and each account has 1 test patient with 1 completed assessment. The participants have not used the system before and receive no training.

Input: 5 clinicians are each asked to locate and use 5 core functions, one at a time, by name only: add a note to a question, edit a result, open questions in the SAFA tool, generate a report, and export annotations.

Output: For each core function, at least TASK_SUCCESS_RATE% of the clinicians locate and use it on their first attempt, relying only on the labels shown on screen and without assistance NFR2.

How the test will be performed:

1. Name each function to the clinician without describing where it is or what its button looks like.
2. Record whether the clinician's first click or selection leads to the correct function.
3. For each function, calculate the percentage of clinicians who succeeded on their first attempt and compare it to TASK_SUCCESS_RATE%.

3. NFR-ST-U3: **Return to a Previous Workflow Stage**

Purpose: Verifies that clinicians can return to any previously visited stage of their workflow with minimal effort.

Type: Nonfunctional, Dynamic, Manual

Initial State: A clinician account is logged in, and a patient record exists with a completed assessment that has a transcript.

Input: The tester visits the workflow stages in order: patient

list, assessment, question, and transcript. From the transcript, the tester returns to each of the 3 earlier stages, and from the question, returns to each of the 2 earlier stages.

Output: Each previously visited stage is reached in at most MAX_NAV_STEPS navigation steps from the current page NFR3.

How the test will be performed:

1. Visit the stages in order, then return to each earlier stage using only the system's interface (not the browser's back button), counting each click or selection as one navigation step.
2. Repeat from the question stage.
3. Check that every return took at most MAX_NAV_STEPS steps.

4. NFR-ST-U4: **Show the Result of Each Change**

Purpose: Verifies that the system shows clinicians the result of each change they submit right away.

Type: Nonfunctional, Dynamic, Automatic

Initial State: A clinician account is logged in, and a patient record exists with a completed assessment that has a transcript.

Input: 4 types of changes, each submitted 10 times by an automated browser test: edit a transcript segment, change a question's result, add a note to a question, and add a patient.

Output: For every submitted change, the updated information is displayed within SYSTEM_FEEDBACK_TIME seconds of submission NFR4.

How the test will be performed:

1. Run the automated test, which submits each change and records the time from submission until the updated information appears on screen.
2. Check that all 40 recorded times are at most SYSTEM_FEEDBACK_TIME seconds, and report the longest time for

each type of change.

4.2.2 Understandability

This section verifies that clinicians are able to distinguish between content that was generated automatically and content that has been verified or edited by a clinician.

1. NFR-ST-UN1: **Identify Automated and Verified Content**

Purpose: Verifies that the system makes it clear which content was generated automatically and which has been verified or edited by a clinician.

Type: Nonfunctional, Dynamic, Manual

Initial State: The system is running with a test clinician account for each participant, containing assessments that include both unverified automated content and clinician-verified content. The participants have not used the system before and receive no training.

Input: 3 screens (the assessment overview, the transcript, and the results view), each containing 10 items such as transcript segments, scores, and flags, of which 5 are unverified automated content and 5 are clinician-verified. An answer key lists the status of each item. 5 clinicians who are new to the system take part.

Output: At least TASK_SUCCESS_RATE% of the clinicians correctly identify the status of every item on all 3 screens without assistance NFR14.

How the test will be performed:

1. Show each clinician the 3 screens and ask them to mark each item as either automated or clinician-verified, without explaining how the system shows this.
2. Compare each clinician's answers to the answer key.
3. Calculate the percentage of clinicians who identified every item correctly and compare it to TASK_SUCCESS_RATE%.

4.2.3 Performance

This section verifies that clinicians can review assessments without noticeable waiting, and that the transcript stays synchronized with the recording for recordings of the maximum length. The tests use test assessments A and B described in Section 4.1.5, and are run by a script that controls the browser and records times, since the delays being measured are too short to time reliably by hand.

1. NFR-ST-PF1: **Assessment Overview Load Time**

   Purpose: Verifies that the assessment overview loads quickly enough on a typical connection.

   Type: Nonfunctional, Dynamic, Automatic

   Initial State: A clinician account is logged in, 3 assessments with results and flags exist for a test patient, and the browser's network speed is limited to MIN_BANDWIDTH.

   Input: The overview of each of the 3 assessments, loaded 5 times each with the browser cache cleared before each load (15 loads in total).

   Output: Every load shows the overview's results and flags within OVERVIEW_LOAD_TIME seconds NFR12.

   How the test will be performed:

   1. Clear the browser cache and open the overview of the first assessment.
   2. Record the time from opening the overview until its results and flags are fully shown.
   3. Repeat steps 1–2 for the remaining loads.
   4. Report the longest and average load times, and compare the longest to OVERVIEW_LOAD_TIME.

2. NFR-ST-PF2: **Transcript Synchronization for a Full-Length Recording**

Purpose: Verifies that the highlighted segment stays synchronized with the recording from start to finish, for a recording of the longest supported length.

Type: Nonfunctional, Dynamic, Automatic

Initial State: A clinician account is logged in, and test assessment B is open in the SAFA tool with playback at the start of the recording.

Input: Test assessment B, played from start to finish without pausing.

Output: For every segment, the difference between the playback position when the segment becomes highlighted and the segment's stored start time does not exceed SYNC_TOL milliseconds NFR13.

How the test will be performed:

1. Start playback, and have the script record the playback position each time the highlighted segment changes.
2. When playback ends, compare each recorded position with the stored start time of the segment that was highlighted.
3. Report the largest difference and compare it to SYNC_TOL.

3. NFR-ST-PF3: **Playback Start After Selecting a Segment**

Purpose: Verifies that playback starts quickly and at the right position after a clinician selects a segment, including segments in parts of the recording that have not been loaded yet.

Type: Nonfunctional, Dynamic, Automatic

Initial State: A clinician account is logged in, test assessment B is open in the SAFA tool, and the browser's network speed is limited to MIN_BANDWIDTH. Since NFR13 does not specify a connection speed, the same minimum connection as NFR-ST-PF1 is used, as the recording is streamed from cloud storage.

Input: 20 segments spread across test assessment B, selected in an order that jumps between the start and end of the recording,

so that many selected segments have not been loaded yet.

Output: For each selection, playback begins within SEEK_TIME seconds, at a position within SYNC_TOL milliseconds of the segment's stored start time NFR13.

How the test will be performed:

1. Select the first listed segment, and record the time from the selection until playback begins and the position where it begins.
2. Repeat step 1 for the remaining segments.
3. Report the longest and average times until playback begins, and compare the longest to SEEK_TIME.
4. Compare each starting position to the segment's stored start time, and check that every difference is within SYNC_TOL.

4.2.4 Convenience

This section verifies that common editing actions respond quickly, that clinicians can continue their work on another device, and that clinicians do not need to repeat actions or re-enter information the system already has. These tests use test assessment A described in Section 4.1.5.

1. NFR-ST-CV1: **Response Time for Common Editing and Export Actions**

Purpose: Verifies that common transcript editing and annotation export actions complete within an acceptable time.

Type: Nonfunctional, Dynamic, Automatic

Initial State: A clinician account is logged in, and test assessment A is open in the SAFA tool.

Input: 50 actions performed by a script, 10 of each: saving a text edit, marking a segment as verified, assigning a speaker role, changing a language tag, and exporting the annotations. Generating the transcript is not included, since it is a processing-intensive task.

Output: At least AVG_INTERACTIONS% of the 50 actions complete within INTERACTION_TIME seconds NFR19.

How the test will be performed:

1. Run the script, recording the time from each action until the system shows its result.
2. Count the actions that completed within INTERACTION_TIME.
3. Compute the percentage (actions within INTERACTION_TIME / 50) and compare it to AVG_INTERACTIONS%, and report the slowest type of action.

2. NFR-ST-CV2: **Continue Work on a Second Device**

Purpose: Verifies that a clinician can continue the same annotation work on a different device through a web browser, without transferring any files.

Type: Nonfunctional, Dynamic, Manual

Initial State: The same clinician account is logged in on MIN_SUPPORTED_DEVICES different computers, each using a supported web browser, and test assessment A has no edits.

Input: 3 transcript edits made and saved on the first computer, then 2 more made and saved on the second computer. Each edit is documented with its segment and content.

Output: All 5 edits are shown on both computers after the assessment is reopened, and no file is transferred between them NFR20.

How the test will be performed:

1. On the first computer, make and save the 3 listed edits.
2. On the second computer, open test assessment A and check that the 3 edits are shown.
3. On the second computer, make and save the 2 remaining edits.

4. On the first computer, reopen the assessment and check that all 5 edits are shown.

3. NFR-ST-CV3: **Unsaved Changes Handled Between Stages and Known Fields Pre-Filled**

Purpose: Verifies that unsaved changes are not lost when the clinician switches workflow stages, since the system asks whether to save or discard them, and that the system fills in information it already has.

Type: Nonfunctional, Dynamic, Manual

Initial State: A clinician account is logged in, and test assessment A is open in the SAFA tool with no edits.

Input: 3 transcript edits and 1 annotation, made without selecting save, followed by a switch to the evaluation stage. When the system asks whether to save or discard the changes, save is chosen, and the clinician then returns to the SAFA tool. Then each screen that shows the patient's name or the assessment date is opened.

Output: The system asks whether to save or discard the changes before switching stages. After choosing save and returning to the SAFA tool, all 3 edits and the annotation are shown without the clinician having to make them again. On every screen that uses the patient's name or the assessment date, they are already filled in and match the patient record NFR21.

How the test will be performed:

1. Make the listed edits and annotation without selecting save.
2. Switch to the evaluation stage, and check that the system asks whether to save or discard the changes.
3. Choose save, then return to the SAFA tool.
4. Check that each edit and the annotation are shown.

5. Open each screen that uses the patient's name or the assessment date, and check that they are filled in and match the patient record.

4.2.5 Accuracy

This section verifies that the system is accurate enough for clinicians to rely on when reviewing assessments.

1. NFR-ST-AC1: **Flag Accuracy**

Purpose: Verifies that the system's interference flags are accurate enough for clinicians to rely on.

Type: Nonfunctional, Dynamic, Manual

Initial State: The system is running with a test clinician account for each participant, and each account has 1 test patient with 1 completed assessment. The participants have not used the system before and receive no training.

Input: 3 assessment videos, 2 of which contain interference (e.g., a guardian giving a hint during a question) and the remaining contain none. For each video, an answer key lists the question number and timestamp of each interference, created by the team before testing by watching the assessment.

Output: In the Individual Assessment View, every question with detected interference is visibly marked NFR11. Additionally, the system must:

- Flag at least FLAG_DETECTION_RATE% of the moments the answer key identifies as an interference
- Have no more than FLAG_FALSE_ALARM_RATE% of its flags as false alarms
- Place each flag within FLAG_TIME_TOLERANCE seconds of when the interference occurs in the video.

How the test will be performed:

1. For each video, open the Individual Assessment View.
2. Compare the flagged questions against the answer key of true interference moments. Count a moment as detected if its question is flagged and the flag is within FLAG_TIME_TOLERANCE seconds of the timestamp listed in the answer key, and as missed otherwise. Any flag that is not listed in the answer key counts as a false alarm.
3. Compute the detection rate and false alarm rate over all videos and compare them to FLAG_DETECTION_RATE% and FLAG_FALSE_ALARM_RATE%.

4.2.6 Reliability

This section verifies that the system reliably generates transcripts for assessment videos and keeps clinicians' saved edits and annotations across sessions. This allows clinicians to depend on the system and ensure that their work is not lost.

1. NFR-ST-RE1: **Reliable Transcript Generation**

Purpose: Verifies that the system generates transcripts for assessment videos without the clinician having to restart the process manually.

Type: Nonfunctional, Dynamic, Manual

Initial State: The system is running with a test clinician account for each participant, and each account has 1 test patient with 1 completed assessment. The participants have not used the system before and receive no training.

Input: 20 valid assessment videos.

Output: For at least GENERATE_SUCCESS% of the 20 videos, a transcript is generated as specified in FR1 without the clinician having to restart the process manually NFR17.

How the test will be performed:

1. For each video assessment, open it up in the annotation in-

terface and auto-generate a transcription.

2. When processing finishes, record whether a transcript was generated.
3. Repeat steps 1 - 2 for the remaining videos.
4. Compute the success rate (videos with a transcript / 20) and compare it to GENERATE_SUCCESS%.

2. NFR-ST-RE2: **Preserve Saved Edits and Annotations Across Sessions**

Purpose: Verifies that saved transcription edits and annotations remain available and unchanged after each clinician session.

Type: Nonfunctional, Dynamic, Manual

Initial State: The system is running with a test clinician account for each participant, and each account has 1 test patient with 1 completed assessment. The participants have not used the system before and receive no training.

Input: 3 assessment videos where for each, 5 transcription edits and 5 annotations are made and saved (30 items in total). Each item is documented with the transcript chunk it was made on and its exact content for reference.

Output: For at least SAVE_SUCCESS% of the 30 saved items, the item is still shown and unchanged after the clinician logs out, logs back in, and reopens the assessment NFR18.

How the test will be performed:

1. For each video, make the listed edits and annotations and save them.
2. Log out of the clinician account, then log back in.
3. Reopen the assessment and check each listed item then record whether it is still shown and remains unchanged.

4. Compute the success rate (items shown unchanged / 30) and compare it to SAVE_SUCCESS%.

4.2.7 Recording Quality

The following test case verifies that all recording quality for speech video assessments are met

1. NFR-ST-RQ1: **Only Accept Videos Meeting Minimum Quality**

Purpose: Verifies that the system does not allow an assessment to proceed when the video resolution or audio level is below MIN_VIDEO_RESOLUTION or MIN_AUDIO_LEVEL, respectively.

Type: Nonfunctional, Dynamic, Manual

Initial State: The system is running with a test clinician account for each participant, and each account has 1 test patient with 1 completed assessment. The participants have not used the system before and receive no training.

Input: 3 attempts to start an assessment, each with the camera and microphone set up as follows:

- Attempt 1: video resolution below MIN_VIDEO_RESOLUTION, audio at or above MIN_AUDIO_LEVEL.
- Attempt 2: audio level below MIN_AUDIO_LEVEL, video resolution at or above MIN_VIDEO_RESOLUTION.
- Attempt 3: video resolution exactly MIN_VIDEO_RESOLUTION and audio level exactly MIN_AUDIO_LEVEL.

Output: In Attempts 1 and 2 the assessment cannot proceed, and the system shows the reason (resolution too low for attempt 1, audio level too low for attempt 2). In Attempt 3 the assessment proceeds NFR10.

How the test will be performed:

1. Set the camera and microphone as described for Attempt 1, try to start the assessment, and record whether it proceeds and the reason shown if it does not.
2. Repeat step 1 for Attempts 2 and 3.
3. Check that Attempts 1 and 2 are blocked with the correct reason, and that Attempt 3 proceeds.

4.2.8 Maintainability

This section verifies that patient information and assessment recordings are backed up regularly and can be restored from a backup, so that clinicians' work is not lost.

1. NFR-ST-M1: **Regular Backups**

Purpose: Verifies that the system backs up patient information and assessment recordings, including recent changes, within one BACKUP_INTERVAL.

Type: Nonfunctional, Dynamic, Manual

Initial State: The system is running with a test clinician account for each participant, and each account has 1 test patient with 1 completed assessment. The participants have not used the system before and receive no training.

Input: One mock video assessment conducted on the system, with one edit made to one of its annotations and saved.

Output: After one BACKUP_INTERVAL, a backup for the mock assessment recording exists, including the edited annotation for the assessment transcript NFR9.

How the test will be performed:

1. Conduct the mock video assessment on the system.
2. Edit one annotation, save it, and note the time.
3. Wait one BACKUP_INTERVAL.

4. Open the backup storage and check that a backup exists and contains the recording and the edited annotation.

2. NFR-ST-M2: **Restore from Backup**

Purpose: Verifies that patient information and assessment recordings can be restored from a backup and match the original data.

Type: Nonfunctional, Dynamic, Manual

Initial State: The system is running with a test clinician account for each participant, and each account has 1 test patient with 1 completed assessment. The participants have not used the system before and receive no training.

Input: The backup from NFR-ST-M1, and the mock assessment deleted from the system before restoring.

Output: After restoring from the backup, the mock assessment's recording and edited annotation are the same as before the deletion NFR9.

How the test will be performed:

1. Open the mock assessment and document the recording length and the exact text of the edited annotation.
2. Delete the mock assessment from the system.
3. Restore the system from the backup.
4. Open the restored assessment and check that the recording plays and has the same length, and that the annotation text is the same as what was documented in step 1.

4.2.9 Security

This section verifies that patient data is protected. Clinicians can only access their own patients, idle sessions time out, data is encrypted in transit and in storage, and all access to patient data is logged.

1. NFR-ST-S1: **Restrict Access to Assigned Patients**

Purpose: Verifies that a clinician cannot access another clinician's patients through the interface, by URL, or through the API.

Type: Nonfunctional, Dynamic, Automatic

Initial State: The test database contains two clinicians, each with 2 assigned patients that have at least one assessment. The patient and assessment IDs are documented prior to testing for later reference.

Input: While logged in as the first clinician, 3 requests for each of the second clinician's patients: opening the patient's page by URL, requesting the patient record through the API, and requesting one of the patient's recordings through the API.

Output: All requests are denied with an access error, and no patient data, transcripts, or recordings belonging to the second clinician are returned (NFR5).

How the test will be performed:

1. Using an automated test, load the data from the test database and log in as the first clinician.
2. Send each request in the input list using the second clinician's documented IDs.
3. Check that every request returns an access error and that no response contains the second clinician's patient data.

2. NFR-ST-S2: **Session Timeout After Inactivity**

Purpose: Verifies that an idle session expires, the clinician's work is saved, and the clinician must log in again before any patient data is shown.

Type: Nonfunctional, Dynamic, Manual

Initial State: A clinician account is logged in, and an assessment is open with one unsaved transcript edit.

Input: No activity in the browser for IDLE_TIMEOUT minutes, then an attempt to open another patient record.

Output: After IDLE_TIMEOUT minutes, the system saves the clinician's work, returns them to the login page with a message explaining the session ended due to inactivity, and shows no patient data until they log in again (NFR6).

How the test will be performed:

1. Make one transcript edit without saving, and note the time.
2. Leave the browser idle for IDLE_TIMEOUT minutes, then check that the login page is shown with the inactivity message.
3. Try to open another patient record and press the browser's back button, and check that no patient data is shown in either case.
4. Log back in, reopen the assessment, and check that the transcript edit was saved.

3. NFR-ST-S3: **Encrypt Data in Transit**

Purpose: Verifies that recordings, transcripts, and patient data are encrypted using TRANSIT_ENCRYPTION_STANDARD when sent between the browser and the server, and from the server to third-party transcription services.

Type: Nonfunctional, Dynamic, Manual

Initial State: A clinician account is logged in, and a network capture tool is running on the test device.

Input: 5 actions performed while traffic is captured: logging in, opening a patient record, playing a recording, downloading a report, and submitting a recording for transcription.

Output: All captured traffic uses and requests to the transcription service use TRANSIT_ENCRYPTION_STANDARD, and no patient data, credentials, or recordings appear in plain text (NFR7).

How the test will be performed:

1. Start the capture and perform each action in the input list.

2. Stop the capture and check the protocol and version used by each connection.
3. Search the capture for the patient's name and the clinician's email, and check that neither appears in plain text.
4. Try to open the site over unencrypted HTTP and check that the connection is redirected to an encrypted one or refused.
5. Check the server logs for the transcription request and confirm it was sent using TRANSIT_ENCRYPTION_STANDARD.

4. NFR-ST-S4: **Encrypt Stored Data**

Purpose: Verifies that stored recordings and transcripts are encrypted using STORAGE_ENCRYPTION_STANDARD.

Type: Nonfunctional, Static and Dynamic, Manual

Initial State: One mock assessment with a recording and transcript has been saved to the system.

Input: The database and file storage configuration, and the raw stored files for the mock assessment.

Output: The storage configuration shows STORAGE_ENCRYPTION_STANDARD is enabled for the database and file storage, and the raw stored recording and transcript cannot be read outside the system (NFR7).

How the test will be performed:

1. Review the storage configuration and record the encryption setting for each storage location.
2. Access the raw stored files for the mock assessment directly, outside the system.
3. Check that the recording cannot be played and the transcript text cannot be read.

5. NFR-ST-S5: **Log Access to Patient Data**

Purpose: Verifies that every access to patient data is recorded in an audit log that clinicians cannot edit.

Type: Nonfunctional, Static and Dynamic, Automatic and Manual

Initial State: A clinician account is logged in, and a patient record with one assessment exists.

Input: 4 actions on the patient's assessment: viewing it, editing one transcript segment, downloading the recording, and exporting the report. The time of each action is documented for later reference.

Output: The audit log contains one entry for each action, with the user, the action, the item affected, and the date and time. Attempts by the clinician to edit or delete entries are denied (NFR8).

How the test will be performed:

1. Using an automated test, perform each action in the input list as the clinician.
2. Check that the audit log has a matching entry for each action, with the correct user, action, item, and time.
3. While logged in as the clinician, try to edit and delete a log entry through the interface and the API, and check that both are denied.

4.2.10 Compliance

This section verifies that TeLaC meets the HIPAA Security Rule technical safeguards. Both tests are static and done through document review.

1. NFR-ST-CO1: **HIPAA Technical Safeguards Review**

Purpose: Verifies that each HIPAA Security Rule technical safeguard is covered by a mechanism in TeLaC.

Type: Nonfunctional, Static, Manual

Initial State: The system design and the tests in the Security

section are complete.

Input: A table mapping each technical safeguard (access control, audit controls, integrity, person or entity authentication, and transmission security) to the TeLaC mechanism and test that covers it.

Output: Every safeguard is mapped to at least one implemented mechanism and one passing test, with no gaps (NFR15).

How the test will be performed:

1. Fill in the mapping table for each technical safeguard.
2. Review each row and record any safeguard without a mechanism or passing test as a GitHub issue.
3. Share the completed table with the security supervisor for review, and record their feedback as GitHub issues.
4. Check that all issues are closed and every safeguard is covered.

2. NFR-ST-CO2: **Business Associate Agreements for Third-Party Services**

Purpose: Verifies that every third-party service that handles patient health information has a Business Associate Agreement (BAA).

Type: Nonfunctional, Static, Manual

Initial State: The list of third-party services used by TeLaC is finalized.

Input: The list of third-party services, including Whisper and Deepgram, and each provider's BAA documentation.

Output: Every service that stores, processes, or transmits patient health information has a BAA in place, and any service without one is not used with real patient data (NFR15).

How the test will be performed:

1. List every third-party service and record whether it handles

patient health information.

2. For each service that does, check whether a BAA is in place and record the result.
3. Record any service without a BAA as a GitHub issue.

4.2.11 Capacity

This section verifies that the system can store a full clinical caseload for every clinician without seeing any deterioration in system performance.

1. NFR-ST-CA1: **Store a Full Caseload**

Purpose: Verifies that the system stores the patients, assessments, recordings, and annotations of a full caseload for every clinician and still meets its usability and performance requirements.

Type: Nonfunctional, Dynamic, Automatic

Initial State: The system is running in a test environment with no existing data.

Input: A script generates NUM_CLINICIANS clinician accounts, each with CASELOAD patients, and each patient with ASMTS_PER_PT completed assessments. Each assessment has a recording of up to MAX_REC_LEN minutes, a transcript, and annotations.

Output: Every generated patient, assessment, recording, transcript, and annotation is stored and can be opened, and NFR-ST-U4 and NFR-ST-PF2 still pass with this data loaded NFR16.

How the test will be performed:

1. Run the script to load the data.
2. Check that the number of stored records of each type matches the number generated.
3. Open a random sample of 20 assessments and check that each recording plays and its transcript and annotations are shown.

4. Run NFR-ST-U4 and NFR-ST-PF2 with the data loaded and check that both pass.

4.3 Traceability Between Test Cases and Requirements

The following tables show which requirements in the SRS each system test verifies. Every functional and nonfunctional requirement is covered by at least one test. Some functional tests trace to a use case rather than a functional requirement, since the SRS describes some behaviour, such as logging in, creating an account, or marking a recording as reviewed, only through use cases; these tests are shown in Table 4.

Table 3: Traceability Between Functional System Tests and Functional Requirements

Test Case	FR1	FR2	FR3	FR4	FR5	FR6	FR7	FR8	FR9	FR10	FR11	FR12	FR13	FR14	FR15	FR16	FR17	FR18	FR19	FR20	FR21	FR22	FR23	FR24	FR25	FR26
FR-ST-DCS1																			X							
FR-ST-DCS2			X																							
FR-ST-VAA1	X																									
FR-ST-VAA2				X																						
FR-ST-VAA3													X													
FR-ST-VAA4																										X
FR-ST-PM1									X																	
FR-ST-PM2									X																	
FR-ST-PM3										X																
FR-ST-PM4											X															
FR-ST-TR1		X																								
FR-ST-TR2						X																				
FR-ST-TR3						X																				
FR-ST-TR4															X											
FR-ST-PN1																				X						
FR-ST-PN2																					X					
FR-ST-PN3														X												
FR-ST-PN4					X																					
FR-ST-AE1																X										
FR-ST-AE2																	X									
FR-ST-AE3																		X								
FR-ST-AE4								X																		
FR-ST-AE5								X																		
FR-ST-RD1												X														
FR-ST-RD2																							X			
FR-ST-RD3																									X	
FR-ST-RD4							X																			
FR-ST-RD5																						X				
FR-ST-AI1																								X		

Table 4: Traceability Between Functional System Tests and Use Cases

Test Case	UC1	UC2	UC3	UC5	UC8	UC9	UC10	UC12	UC13	UC14
FR-ST-DCS3							X			
FR-ST-DCS4										X
FR-ST-A1								X		
FR-ST-A2								X		
FR-ST-A3								X		
FR-ST-A4	X									
FR-ST-A5	X									
FR-ST-A6		X								
FR-ST-PM3			X							
FR-ST-PM4									X	
FR-ST-TR1						X				
FR-ST-TR2				X						
FR-ST-PN1					X					

Table 5: Traceability Between Nonfunctional System Tests and Nonfunctional Requirements

Test Case	NFR1	NFR2	NFR3	NFR4	NFR5	NFR6	NFR7	NFR8	NFR9	NFR10	NFR11	NFR12	NFR13	NFR14	NFR15	NFR16	NFR17	NFR18	NFR19	NFR20	NFR21
NFR-ST-U1	X																				
NFR-ST-U2		X																			
NFR-ST-U3			X																		
NFR-ST-U4				X																	
NFR-ST-UN1														X							
NFR-ST-PF1												X									
NFR-ST-PF2													X								
NFR-ST-PF3													X								
NFR-ST-CV1																			X		
NFR-ST-CV2																				X	
NFR-ST-CV3																					X
NFR-ST-AC1											X										
NFR-ST-RE1																	X				
NFR-ST-RE2																		X			
NFR-ST-RQ1										X											
NFR-ST-M1									X												
NFR-ST-M2									X												
NFR-ST-S1					X																
NFR-ST-S2						X															
NFR-ST-S3							X														
NFR-ST-S4							X														
NFR-ST-S5								X													
NFR-ST-CO1															X						
NFR-ST-CO2															X						
NFR-ST-CA1				X									X			X					

5 Unit Test Description

[This section should not be filled in until after the MIS (detailed design document) has been completed. —SS]

[Reference your MIS (detailed design document) and explain your overall philosophy for test case selection. —SS]

[To save space and time, it may be an option to provide less detail in this section. For the unit tests you can potentially layout your testing strategy here. That is, you can explain how tests will be selected for each module. For instance, your test building approach could be test cases for each access program, including one test for normal behaviour and as many tests as needed for edge cases. Rather than create the details of the input and output here, you could point to the unit testing code. For this to work, you code needs to be well-documented, with meaningful names for all of the tests. —SS]

5.1 Unit Testing Scope

[What modules are outside of the scope. If there are modules that are developed by someone else, then you would say here if you aren't planning on verifying them. There may also be modules that are part of your software, but have a lower priority for verification than others. If this is the case, explain your rationale for the ranking of module importance. —SS]

5.2 Tests for Functional Requirements

[Most of the verification will be through automated unit testing. If appropriate specific modules can be verified by a non-testing based technique. That can also be documented in this section. —SS]

5.2.1 Module 1

[Include a blurb here to explain why the subsections below cover the module. References to the MIS would be good. You will want tests from a black box perspective and from a white box perspective. Explain to the reader how the tests were selected. —SS]

1. test-id1

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

Input:

Output: [The expected result for the given inputs —SS]

Test Case Derivation: [Justify the expected value given in the Output field —SS]

How test will be performed:

2. test-id2

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

3. ...

5.2.2 Module 2

...

5.3 Tests for Nonfunctional Requirements

[If there is a module that needs to be independently assessed for performance, those test cases can go here. In some projects, planning for nonfunctional tests of units will not be that relevant. —SS]

[These tests may involve collecting performance data from previously mentioned functional tests. —SS]

5.3.1 Module ?

1. test-id1

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input/Condition:

   Output/Result:

   How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input:

   Output:

   How test will be performed:

5.3.2 Module ?

...

5.4 Traceability Between Test Cases and Modules

[Provide evidence that all modules have been considered. —SS]

6 Appendix

This appendix provides supporting information for the previous sections.

6.1 Symbolic Parameters

The definition of the test cases will call for SYMBOLIC_CONSTANTS. Their values are defined in this section for easy maintenance.

6.2 Requirements Feedback Form

The following form guides the supervisor and research team review that is part of the SRS verification (Section 3.2). The form description links to the functional requirements and states that features of the existing recording platform, built by the previous capstone team, are not included.

1. Do the requirements accurately reflect how you would ideally review a remote speech-language assessment? If not, please explain why.
2. Are there any features or capabilities missing from the current requirements that you believe the system should support?
3. Are there any requirements that you believe should be changed, clarified, or removed?
4. Which requirements are the most important, or the ones we should prioritize for the initial demo (top 10–15)?
5. Are there any requirements that you consider essential for the system to be usable in a clinical setting?
6. How should the automatically generated transcript (that SAFA outputs) be shown in the clinical assessment interface? (Choose one)
    - Display the transcript beside the video in the main clinical assessment view by default
    - Display it beside the video, but only when the clinician chooses to open it (it can be hidden)
    - Display the transcript only when the clinician opens the SAFA tool

Variable Name	Value
MIN_VIDEO_RESOLUTION	640 x 480 pixels
MIN_AUDIO_LEVEL	-30 dBFS average
GENERATE_SUCCESS	95
SAVE_SUCCESS	100
FLAG_DETECTION_RATE	80%
FLAG_FALSE_ALARM_RATE	20%
FLAG_TIME_TOLERANCE	2 seconds
BACKUP_INTERVAL	24 hours
IDLE_TIMEOUT	15 minutes
TRANSIT_ENCRYPTION_STANDARD	TLS 1.2 or higher
STORAGE_ENCRYPTION_STANDARD	AES-256
MIN_PASSWORD_LENGTH	8 characters
TASK_SUCCESS_RATE	80%
TASK_TIME	5 minutes
MAX_NAV_STEPS	3 steps
SYSTEM_FEEDBACK_TIME	2 seconds
NUM_CLINICIANS	10 clinicians
CASELOAD	30 patients
ASMTS_PER_PT	10 assessments
MAX_REC_LEN	30 minutes
OVERVIEW_LOAD_TIME	3 seconds
MIN_BANDWIDTH	25 Mbps
SYNC_TOL	500 ms
SEEK_TIME	2 seconds
AVG_INTERACTIONS	95%
INTERACTION_TIME	2 seconds
MIN_SUPPORTED_DEVICES	2 devices

Table 6: Variable Names and Values

- Other

7. What information should the system use to generate a clinical report for an assessment? (Select all that apply)

    - Reviewed transcript
    - Questions

- Question responses
- Clinician notes per question
- Clinician notes for the whole assessment
- All flagged potential interference
- Final diagnosis/results
- Other

8. Who should be allowed to download or view recordings? (Select all that apply)
    - Clinicians registered in the system
    - Parents registered in the system (only their child's recordings)
    - Researchers who have access to the system
    - Other
9. Which pattern detection features should be in the scope of this project? (Select all that apply)
    - Grammatical errors
    - Code-switching in answers
    - Phonetic errors (target sound accuracy)
    - Other

6.3 Usability Survey Questions

The following survey will be given to each participant right after their usability testing session in the Usability Report CD (Section 2.3). Since the system has not been built yet, this section is a work in progress and will be finalized after the Wireframe review to account for any changes the team might make in response to clinician feedback. Participants will be asked to complete the same survey after doing the same tasks in the original standalone SAFA tool, so the two scores can be compared.

Part A: System Usability Scale Part A uses the System Usability Scale (SUS) (**?**), a standard ten-item questionnaire that produces a single usability score from 0 to 100. This scale was chosen because it is short, widely used, and gives a score that can be compared between the integrated system and the standalone SAFA tool. Each statement is rated from 1 (strongly disagree) to 5 (strongly agree).

1. I think that I would like to use this system frequently.
2. I found the system unnecessarily complex.
3. I thought the system was easy to use.
4. I think that I would need the support of a technical person to be able to use this system.
5. I found the various functions in this system were well integrated.
6. I thought there was too much inconsistency in this system.
7. I would imagine that most people would learn to use this system very quickly.
8. I found the system very cumbersome to use.
9. I felt very confident using the system.
10. I needed to learn a lot of things before I could get going with this system.

Part B: Project-Specific Questions The SUS measures overall ease of use, but does not cover the specific usability requirements that the team defined in the SRS. This part adds statements that target those requirements, rated on the same 1 to 5 scale. These ratings show how participants felt about each requirement, while the task results recorded by the team during the session (completion rate, time, and help needed) are used to check whether the fit criteria are met.

11. I could find each function I needed on my first attempt by reading the labels on the screen (NFR2).

12. I could always tell which transcript content and results were automatically generated and which I had verified (NFR14).

13. I could easily return to an earlier stage without losing my work (NFR3).

14. After I made a change, it was clear that the system had saved it (NFR4).

15. Following the transcript alongside the video playback was easy (FR20).

16. Reviewing, annotating, and reporting on an assessment in one place would save me time compared to my current process (SR1, SR2).

Part C: Open-Ended Questions

17. Which task was the most difficult to complete, and why?

18. Was anything on the web pages confusing or unclear?

19. What would you change or add to make the system fit better into your daily workflow or help streamline your tasks?

20. Is there anything else you would like to tell us about your experience?

7 Appendix — Generative AI Use Disclosure

This section documents all the uses of generative artificial intelligence used throughout the document.

7.1 AI Tools Used

- Claude Sonnet 5 (Medium)
- ChatGPT-5.6 Luna

7.2 How and Where Used

The following lists the uses of AI applied throughout the entire document:

- Fixing grammar and spelling
- Debugging possible LaTeX errors
- Performing a final check over the document at the end, comparing each section against the original document comments and the rubric to ensure nothing was missed
- Creating references for the bibliography
- Removing template comments

Section 2:

- To create commands to link the requirements to the SRS and Hazard Analysis document

Section 4:

- Brainstorming test cases
- Splitting the test cases among the group members
- Creating the traceability matrix

7.3 Verification of AI-Generated Content

To verify the LLM's output, each team member went over their own section and presented it to the group, explaining what they had completed since the previous meeting. Every member was able to explain their section successfully. Before any pull request was approved, a team member who had not written the section also reviewed it to ensure it was cohesive, accurate, and consistent with the rest of the document. Since the document contained no factual content, no additional fact-checking of the LLM's output was needed.

8 Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?
2. What pain points did you experience during this deliverable, and how did you resolve them?
3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.
4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

Nour

Question 1

I wrote the Summary section of this deliverable. At this point, it's very clear what the objectives and core components of the project are. Hence, breaking

it down and determining how it maps back to our SRS was not difficult and was rather automatic. Additionally, considering our project builds on a previous year's capstone, we are using several of the same tools that they used. Hence, it was straightforward as to what frameworks will be needed for automated testing and verification. Moreover, when creating the test cases for the nonfunctional requirements, it was simple to divide them into different "Areas of Testing". This is thanks to the template that we used for the SRS document. It broke down the nonfunctional requirements into subsections making it clear which aspects regarding nonfunctional requirements needed testing.

Question 2

A pain point that the team experienced was splitting the functional requirements into different sections. As mentioned previously, the SRS template encouraged us to organize the nonfunctional requirements into subsections, however, it did not for the functional requirements. At the time this didn't seem like much of a problem, but it did make splitting the FRs into disjoint sections a bit difficult. A lot of the functional requirements map back to one another, so ensuring that the sections did not overlap was difficult. However, given that we are now more familiar with the system, especially after writing the SRS document and creating different components for risk mitigation in the Hazard Analysis document, it wasn't too difficult, mainly just time consuming. They were organized during a team meeting to ensure we all had a similar understanding of the system components and that we were on the same page about the main functions of the system.

Question 3

There are several different aspects that each team member has to be knowledgeable about to ensure we are able to successfully complete the verification and validation aspect of the project. I will personally tackle usability testing. This is because I have prior experience in this domain of testing. First, I had a couple of friends conduct usability tests on a game I created to detect any bugs and to determine what type of actions a user might take that may not come to mind when I am testing the game myself. This actually helped me catch a bug in the game, which I was later able to fix and update in the latest release. Second, I previously took the course SFWRENG 4HC3 which

is the Human Computer Interfaces course at McMaster. In that course, I ran card sorts, online usability interviews, and Figma prototype interviews where I observed users and asked several appropriate questions that avoided any biased wording. However, what I still need to build is applying these methods to a clinical setting, writing realistic SLP tasks, and using a standard questionnaire such as the System Usability Scale to produce scores we can compare against our requirements.

Question 4

The first approach I will use is reviewing and adapting my previous course material for this new application. Although I am familiar with usability testing, it has been a while, and the domains in which I applied the skills are vastly different from this one. Hence, I will review my 4HC3 notes and adapt the interview and observation methods I already used to this project.

The second approach is running mock interview sessions with friends and peers to ensure my wording and intent are clear. I will do this before involving any SLPs because, first there will not be many opportunities to ask the SLPs their opinions about certain features of TeLaC. Second, I do not want to waste their time by asking them questions that are hard to understand and are unclear. Ensuring that anyone can understand the point I am getting across will allow the clinician to quickly comprehend the question and prepare a well-thought-out answer.

I will pursue both approaches. This is because the first approach builds on methods I've already used, which is the fastest way to be effective. As for the second approach, it allows me to practice my interview skills at low risk and, because I have not conducted any usability tests with actual clinicians, it's a good way to develop my skills before being thrown into that setting.

Catherine

Question 1

I wrote the Relevant Documentation, Design Verification, and Software Validation sections, as well as some of the functional and nonfunctional test cases. As we previously had brainstormed together a table with all our requirements, grouped them into specific themes/sections, and came up with corresponding test cases together, it was incredibly easy to split the work

into equal sections between everyone. So everybody more or less was able to have the same amount of work to do. As members started writing their section, we were able to easily review each others' work and also easily copy their structure and formatting, helping us keep the document consistent and cohesive! The SRS also made the work easier, since each requirement already had a fit criterion and most of the thresholds were defined as symbolic constants. This meant that the expected output of each test could come directly from the SRS instead of being decided from scratch.

Question 2

One pain point that we had during this deliverable was that the planned test cases didn't always line up with the requirements. When I compared the tests assigned to me against the SRS, I found that FR22 technically wasn't covered by any test and one of the planned tests didn't actually trace back to any functional requirement. I had to resolve this by replacing that test with one for FR22, so that every requirement in my sections is covered. As we also had to write the test cases before we had any design documents complete, it was a little difficult to describe some of the test cases in a sure way, but writing the test cases in a more general way with additional notes and references that we would update later helped with this process for me personally.

Question 3

Although a variety of testing skills is important for everyone on the team to have, I think given that our project will entail a lot of different modules working together, whilst also being implemented within a web application and cloud environment, it is important for me to learn how to do integration testing. For example, a transcript edit made by a clinician has to be saved, passed to the SAFA analysis, and then reflected in the assessment results and the final report. Each of these steps could pass its own unit test while the overall flow still fails. Personally, I'll ensure that I'm able to learn how to test the interactions between modules, such as between the frontend, the backend, the database, and external services like the speech recognition API, and how to set up data that represents a realistic assessment.

Question 4

I think the first approach to learning how to design comprehensive integration tests, is through documentation and tutorials with the tools we already use, such as how to use pytest for the backend or Jest for the frontend. This also includes learning how to test API endpoints together with the database, and how to replace external services with mock versions so that tests can be repeatable against different modules, saving the team time and effort as we'll potentially have a ton of different tests to create.

The second approach is to learn by doing, so by writing integration tests for one complete flow in our system, whether it's for a mock system or a small isolated scenario. For example, uploading a recording, editing its transcript, and generating a report. Starting with one flow and asking teammates to review the tests will show me where my tests miss problems before I write tests for the rest of the system.

Going forward though, I'll pursue both approaches. Going through the documentation first is the easiest way for me to know that I'm on the right track, and will help me avoid writing tests that are slow or potentially flawed. Since my test cases cover results and reporting, which depend on almost every other part of the system, integration testing is the most useful skill for me to verify that my sections work as intended.

Yilin

Question 1

This deliverable was a lot smoother to complete because of the work we put into the SRS document. Since it was so detailed, we were able to test directly against the functional and nonfunctional requirements. For instance, NFR8's fit criterion already described the expected outputs and the constants it had to meet. Even though I did not personally write every requirement I tested in this deliverable, the clear categories and descriptions in the SRS were very useful.

Question 2

At this point in the project, we have many documents, acronyms, and constants to keep track of, which makes writing new content more time-consuming since I want to reference past documents. The traceability tables

helped a lot with this, almost like a first hint at where in the previous documents to start looking. Also, keeping a list of any new symbolic parameters and references as I wrote meant I wasn't scrambling to edit the table, and I could focus on my sections first.

Question 3

A skill I will personally need to develop is static testing for system integration and security. Several of our security tests are partly or fully static, such as reviewing the storage encryption settings, the log retention configuration, and the HIPAA safeguard mapping. Integrating SAFA with the clinician dashboard also creates new connection points, such as API endpoints and the patient data passed between the two systems, where access-control or data-handling bugs can easily slip in. Many of these are easier to catch by reviewing the code and configuration than by running tests. To do this well, I need to learn how to inspect code and configuration for security problems, use static analysis tools, and review the integration points against our SRS.

Question 4

The first approach is learning static analysis tools for security, such as Bandit for Python, which scan code for issues like hardcoded secrets, missing access checks, and unsafe data handling. I would learn them through their documentation and tutorials. These tools can also be added to our GitHub Actions workflow, so the checks run automatically on every pull request. The second approach is practising static review on the existing software. Since TeLaC builds on the previous capstone's TeleHealth platform and the SAFA tool, I can review their code and configuration before our integration is complete. This includes checking whether their API endpoints verify that the requested patient belongs to the logged-in clinician, how recordings are stored, whether storage encryption is enabled, and what activity is logged. Any issues I find would be recorded as GitHub issues. This gives me hands-on practice on the actual system we are integrating.

I will pursue both approaches. Static analysis tools catch common problems automatically with little effort once set up, which matters given our limited time. Reviewing the existing software builds the judgment tools lack, such as whether the system follows our access-control requirements, and it lets me find security gaps before they carry into TeLaC.

Javayria

Question 1

During this deliverable, I found that the template itself was very helpful and provided much needed guidance. Its hints made it clear what a structured review should include, such as who reviews the document, what questions are asked, and how feedback is tracked. While writing the SRS Verification section, I realized that our team was already following most of what a checklist would cover when reviewing each other's pull requests, such as checking that requirements trace back to stakeholder needs and that fit criteria can actually be measured. All we had to do was write it out as a formal checklist, so that nothing is forgotten in future reviews and every reviewer checks the same things. Our team was also very good at communicating, which made it easy to coordinate the different sections of work. We split the sections and test areas between members early and worked on them independently, but we also discussed topics that affected more than one person as a group, such as which tests were needed for transcript playback, so that our test cases did not overlap or contradict each other.

Question 2

Something we found difficult with this deliverable was determining realistic tests that would actually verify and validate our requirements, rather than tests that only looked good on paper. Since our tests are built directly from the fit criteria in the SRS, we wanted our supervisors to check the requirements before we started writing the VnV plan in detail. However, we did not receive feedback on our SRS until very late into working on this deliverable. To resolve this, two of our team members asked our supervisors to go over the document together during our scheduled biweekly Zoom meeting, which proved to be much more difficult and time-consuming than we initially thought. We also had to figure out what type of data we would have access to, since real patient recordings cannot be used during development. The research team already had test recordings of family members and friends, so it was important for us to identify what these recordings contained and whether we could use them in our test cases. Knowing this ended up making our tests more specific, since each test could describe exactly which recording it uses and what results to expect from it.

Question 3

Since my test cases focus on transcript playback and performance, the main skill I will need is automated end-to-end testing in the browser. Several of my tests, such as checking that the transcript stays synchronized with the video and that playback starts quickly after a segment is selected, measure delays of only a few hundred milliseconds, which cannot be timed reliably by hand. These tests need a script that controls the browser, reads the video's playback position, checks which transcript segment is highlighted, and limits the network speed to simulate a slower connection. This is quite different from the unit tests the team will write for individual modules, since it involves the whole system running together, including video streamed from cloud storage. I will also need a better understanding of our continuous integration pipeline. While working on this deliverable, our document build on GitHub started failing because of a setup step that the workflow was missing, which showed me that automated checks are only useful if the team understands how they work and can fix them when they break.

Question 4

The first approach I will use is working through the official documentation and tutorials for a browser testing framework such as Playwright, focusing on the features my tests rely on, such as reading the state of a video element, waiting for an element to change, and limiting network speed. The second approach is writing small practice tests against the existing TeleHealth review page, which already plays assessment videos, before the SAFA integration is built. This would let me find out early whether measurements like playback position can be read accurately in an automated test, rather than finding out once the system is nearly complete. I will pursue both approaches. The documentation gives me a reliable starting point and shows the intended way to use the tool, while the practice tests let me apply it to our actual system and catch problems early. Since the timing thresholds in my tests are strict, making sure the measurement method itself works is just as important as the system meeting them.